
THE FORMAL CLOSURE OF COMPUTATIONAL SEPARATION: $P \neq NP$ CLOSED ON EXISTENCE ITSELF, READ ON COMPUTATION

A Lean 4 Formalization via RAcomp, Least Escape, and the Barrier-Evasion Frontier

COMPUTATIONAL COMPLEXITY

One Posit Carries the Whole Separation

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent Researcher, United States

2026-10-02

ABSTRACT. The P versus NP problem has resisted every method, and relativization, natural proofs and algebrization prove that whole families of arguments cannot separate the classes. This paper anchors the problem in existence and closes it on one posit. Existence and freedom are the ground: the Root Axiom is satisfiable, every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation, the problem's native involution, which has no fixed point. The separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives it on no axiom. The root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, on no axiom, and it is the one place where existence meets the separation: no statement that reads the same on every machine, not existence, not freedom, not the arrow, is that reading. The reading is the single posit, the weakest premise on which the separation closes, and $P \neq NP$ holds on it at premise grade. Around the closure stand a double defense, in which nothing passes for the separation but the reading and nothing against it but a computed escape; a held throne, whose holder, Levin's universal search, decides the separation on no axiom; and a frontier that types every route, Williams' route wired to the machine on two named objects. All one hundred eighty theorems are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned and printed.

KEYWORDS P versus NP · computational complexity · existence · the root read on computation · Levin universal search · relativization · natural proofs · algebrization · Williams algorithms to lower bounds · formal verification · Lean 4

1. Introduction

The P versus NP problem asks whether every decision problem whose solutions can be verified in polynomial time can also be solved in polynomial time (Cook 1971; Levin 1973; Karp 1972). Three theorems explain why the principal methods have not settled it: relativization (Baker, Gill and Solovay 1975), natural proofs (Razborov and Rudich 1997) and algebrization (Aaronson and Wigderson 2009). Each proves that a whole family of arguments cannot separate the classes.

This paper anchors the problem in existence and closes it on one posit. The arc has four steps. The ground is existence and freedom: the Root Axiom, that to exist is to actuate, is satisfiable, every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation. The machine comes next: the separation is defined on a machine

whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives the separation on no axiom. The reading is the third step: the root read on computation, every decider of satisfiability actuating beyond every polynomial bound, is the separation itself, and it is the one place where existence meets the separation; the kernel proves that no statement reading the same on every machine, not existence, not freedom, not the arrow, carries it. The closure is the fourth: on the reading, $P \neq NP$ holds, and around it stand a double defense, a held throne and a frontier that types every route.

The single posit is the root read on computation, and every proof of $P \neq NP$ by any method fills it. The one hundred eighty theorems that carry the arc are compiled in Lean 4.19.0 with no import, no custom

axiom and no unproved step, every footprint pinned, and printed in full in the appendices.

1.1 Formal Scope and Machine-Verified Status

Formal Scope and Machine-Verified Status

Unconditional, at theorem grade, compiled in Lean 4.19.0 with no custom axiom. Existence is satisfiable, and existence and freedom as written are conservative (Theorems 1 and 2). The deed instantiates existence at every act (The-

orem 3). Every prime's fibre is a copy of complementation (Theorem 4). The absolute asymmetry (Theorem 5). The closure gives the separation on the machine, on no axiom (Theorem 6). The root read on computation is the separation, on no axiom, and is the closure (Theorems 8 and 9). Nothing keyless is the reading (Theorem 11). The double defense (Theorems 13 to 16). The throne's holder decides the separation, on no axiom (Theorem 17). The vanishing seat (Theorem 19). The frontier and the route (Theorems 21 to 26). The master seal (Theorem 27).

On the root read on computation, at premise grade. The separation $P \neq NP$.

Table 1 | The theorem map: each result and its logical status.

Result	Lean symbol	Logical status
Existence is satisfiable	<code>ra_satisfiable</code>	Unconditional theorem
Existence and freedom add nothing as written	<code>posits_add_nothing</code>	Unconditional theorem
Every prime's fibre is a copy of complementation	<code>prime_fibre_is_complementation</code>	Unconditional theorem
The closure gives the separation	<code>sep_of_closure</code>	Unconditional theorem, on no axiom
The root read on computation is the separation	<code>ra_comp_is_sep</code>	Unconditional theorem, on no axiom
The reading is the closure and the separation	<code>the_three_names_are_one</code>	Unconditional theorem
Nothing keyless is the reading	<code>no_keyless_statement_is_least_escape</code>	Unconditional theorem
The holder decides the separation	<code>the_holder_decides_the_separation</code>	Unconditional theorem, on no axiom
The route closes the machine's separation on its two objects	<code>route_closes_on_the_machine</code>	Unconditional theorem, on no axiom
The separation $P \neq NP$	<code>sep_from_the_act</code>	Holds on the root read on computation, at premise grade

2. Background and Barrier Analysis: Why Prior Methods Are Incomplete

The prior methods fail at three precise places, and each failure is a theorem about a method.

Diagonalization stops at relativization. Diagonalization separates classes defined by resources of different size, and the time hierarchy is its triumph (Hartmanis and Stearns 1965). But it treats machines as black boxes, so it survives the addition of an oracle, and Baker, Gill and Solovay built oracles on which the classes are equal and oracles on which they differ.

Circuit lower bounds stop at natural proofs. Super-polynomial bounds hold in restricted models, monotone circuits (Razborov 1985) and constant-depth circuits (Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986), while for general circuits the strongest explicit bound is linear, $3.1n - o(n)$ (Li and Yang 2022). Razborov and Rudich proved that natural arguments cannot separate the classes if strong pseudorandom functions exist.

Arithmetization stops at algebrization. Arithmetizing arguments pass relativization and algebrize, and algebrizing arguments cannot settle the question (Aaronson and Wigderson 2009).

The common structure, and what this paper does with it. Each barrier is a pair of worlds in which every premise of a method holds. Section 7 proves that the same structure binds every premise that reads the same on every machine, and isolates the one reading that does not.

3. Related Work

The time hierarchy. More time decides more languages (Hartmanis and Stearns 1965); the separations it proves are between resource bounds of different order, by arguments that relativize.

Restricted circuits. Razborov (1985) bounds monotone circuits for clique; Ajtai (1983), Furst, Saxe and Sipser (1984) and Håstad (1986) bound constant-depth circuits for parity. For general circuits the frontier is linear (Li and Yang 2022).

Geometric complexity theory. Mulmuley and Sohoni (2001) recast lower bounds as representation-theoretic obstructions, a program designed to evade the barriers.

Proof complexity. $NP \neq coNP$ holds exactly when no propositional proof system has polynomial-size proofs of all tautologies (Cook and Reckhow 1979).

Query complexity. Black-box search over N candidates needs N tests, and quantum search needs order $\sqrt{N}$ (Bennett, Bernstein, Brassard and Vazirani 1997), attained by Grover (1996).

The physics of computation. Erasing one bit costs at least $k_B T \ln 2$ (Landauer 1961), and every computation can be run reversibly (Bennett 1973).

Algorithms to lower bounds. Savings for satisfiability of a circuit class give NEXP outside that class (Williams 2011), extended to nondeterministic quasipolynomial time (Murray and Williams 2018).

Formal row closures. Islam (2026a) formalizes twenty-three open problems from existence alone, the value excepted by theorem on every row. An earlier paper (Islam 2026f) argued that the structural verdict on P versus NP is the only one left standing. The present paper carries that row to its closure on one posit.

4. Methodology

Every theorem is compiled in Lean 4.19.0 (de Moura and Ullrich 2021) in three kernels that import nothing and declare no axiom, carried by the master seed APEX-PSP-PNP-SEED-03: the seal and its double defense (Appendix B), the frontier of the act (Appendix C), and the fortification on the same machine (Appendix A). Every footprint is pinned by `#guard_msgs`, so a kernel that compiles prints nothing and any change to a footprint fails the build. Each theorem rests on no axiom, or on Lean's standard axioms of propositional extensionality, quotient soundness and choice. The kernels are printed in full with their SHA-256 digests, so any reader reproduces every claim by recompiling.

The architecture that found the arc is Trisduction, whose operating system PhysOS^T judges every cited law under its own negation at each build (Islam 2026c); its register of record and its codex carry the surrounding results (Islam 2026b, 2026d), and the seat and the cut take their form from the Bridge From First Principle (Islam 2026e). The kernels printed here import nothing from it.

5. The Ground: Existence and Freedom

Theorem 1 (Existence is satisfiable). The Root Axiom, read as every state actuating, has a model (`ra_satisfiable`).

Theorem 2 (Existence and freedom are conservative). Any statement derived from the Root Axiom and freedom, uniformly in their interpretation, holds without them (`posits_add_nothing`, `ra_conservative`, `freedom_conservative`). Existence and freedom are the ground of the arc, and as written they constrain nothing outside their own vocabulary.

Theorem 3 (The deed instantiates existence). Every computation re-enacts the root: the pulse

instantiates the Root Axiom at every deed (`deed_instantiates_ra`), in every world (`both_deeds_in_every_world`), and certifies no output (`pulse_does_not_certify`).

Theorem 4 (The prime's triaxis). Freedom is carried by the primes. Every prime's fibre is exactly the two trivial pairs (`prime_fibre`), lies off the multiplicative seat (`prime_off_seat`), and with its swap is the decision bit with complementation, by a bijection carrying the swap to complementation (`prime_fibre_is_complementation`). Every prime carries P versus NP's involution, seatless in the small as the problem is seatless in the large.

Theorem 5 (The absolute asymmetry). What a registration destroys, no procedure of any cost recovers (`destroyed_is_unrecoverable`); what a search hides, exhaustive search recovers (`hidden_is_recoverable`); the two are bound as one statement (`asymmetry_holds`).

6. The Machine and the Separation

A machine has algorithms, instances with a size, each run's output and running time, and the satisfiability of each instance. An algorithm escapes at a bound $c \cdot n^k + c$ when every run is correct and inside the bound. The separation on the machine is that no algorithm escapes at any bound. Nothing escapes when every algorithm, at every bound, meets an instance that stops it, and that closure (least escape) is the one field of a structure.

Theorem 6 (The closure gives the separation). Nothing escaping gives the separation, on no axiom (`sep_of_closure`); the converse holds classically (`closure_of_sep`); and the act carries the separation (`sep_from_the_act`).

Theorem 7 (The socket is the value). The act's socket is the closure itself (`socket_is_the_value`); what the act supplies and what the separation needs are one object.

7. The Reading: The One Place Where Existence Meets the Separation

Theorem 8 (The root read on computation is the separation). Read on computation, the root says that every decider of satisfiability actuates beyond every polynomial bound, and that reading is the separation, on no axiom (`ra_comp_is_sep`).

Theorem 9 (The reading is the closure). The root read on computation, the closure that nothing escapes, and the separation are one statement on one machine (`the_three_names_are_one`).

Theorem 10 (The reading is keyed). The reading holds on some machines and fails on others: on a coherent machine with a constant-time decider it fails

(*ra_comp_is_keyed*), and the closure is keyed likewise (*least_escape_is_keyed*).

Theorem 11 (Nothing keyless is the reading). No statement that reads the same on every machine is the closure the reading states (*no_keyless_statement_is_least_escape*): not prime freedom (*prime_freedom_is_not_least_escape*), not the arrow (*arrow_is_not_least_escape*), not the Root Axiom as written (*ra_is_not_least_escape*). A carrier of the separation must be keyed (*carrier_must_be_keyed*).

Theorem 12 (Every closing premise fills the posit). Every premise from which the separation follows fills the field of the act (*forcing_premise_fills_the_act*) and fills the closure (*every_closing_premise_fills_both*). The reading is the weakest statement on which the separation closes.

Theorems 1 to 12 place the posit exactly. Existence and freedom are the ground and force nothing on their own (Theorems 2 and 11); the root read on computation is the separation (Theorem 8); and that reading is the one meeting point, keyed and irreducible (Theorems 9, 10 and 12).

8. The Double Defense

Theorem 13 (Omega, the formal gate). Nothing passes for the separation but the reading. Every premise either forces the closure or has a twin machine on which it holds and the closure fails (*forces_or_has_a_twin*), and a keyless premise forces nothing (*keyless_forces_nothing*).

Theorem 14 (AEGIS, the actuation gate). Nothing passes against the reading but a computed escape: a rejection of the reading is a witness, an algorithm escaping at a bound (*rejection_is_a_witness*). The root, which every computation re-enacts, crosses neither way (*root_crosses_neither_way*).

Theorem 15 (Every pair lands). Every pair of algorithm and bound a substrate can examine lands on exactly one gate, a stopping instance or a clean escape (*every_pair_lands*). Each examination refutes the escape of the pair it examined (*confirmation_is_atomic*) and pays the floor (*confirmation_pays*); under the act no computation escapes (*act_leaves_no_escape*). The defense is bound whole (*double_defense_hardened*).

Theorem 16 (The finite block). For every stage

n , a coherent machine confirms every examination below n and still escapes at n (*record_decides_nothing*), while a second coherent machine closes whole (*closed_world_closes*). No finite record decides the reading, and the closure is keyed (*armor_scope*).

9. The Throne and the Seat

Theorem 17 (The holder decides the separation). A throne of the machine is a holder that escapes whenever any algorithm escapes. The separation holds exactly when the holder never escapes, on no axiom (*the_holder_decides_the_separation*), and any two holders escape together or not at all, on no axiom (*the_holder_is_unique*). For the satisfiability machine the holder is Levin's universal search, run for decision through the search form by self-reducibility, by Levin's theorem (Levin 1973), carried as the throne's field. Nothing escapes exactly when the holder never escapes (*least_escape_at_the_throne*).

Theorem 18 (The act stops the holder). On the reading the holder meets, at every bound, an instance that stops it (*the_act_stops_the_holder*). The whole of P versus NP is the running time of one explicit program.

Theorem 19 (The vanishing seat). On the locus map between the root and the Riemann seat, the Riemann seat is inhabited (*rh_locus_inhabited*) and the P versus NP seat vanishes (*locus_vanishes*): complementation is involutive with no fixed point, so the ground has dimension zero (*ground_dim_zero*). The carrier halts and the socket opens exactly on the empty configuration (*carrier_halts_iff_empty*, *socket_vanishes*), the vacancy is keyless while the separation is keyed (*vacancy_keyless*, *separation_keyed*), and the three faces lock on one object, the empty seat (*vanishing_gol*).

Theorem 20 (The block chain). The eight absolute barriers of the register are chained, one leg each, and terminated on the empty throne: complementation has no fixed point (*a1_no_seat*), the seat is vacant (*seat_vacant*), the throne is empty (*throne_empty*), and the root forces no value (*root_forces_no_value*), bound as one chain (*block_chain*, *block_chain_terminal*).

The Riemann closure and this one stand in the same two registers and differ at the seat, as Table 3 sets out.

Table 3 | The Riemann closure and the P versus NP closure, step by step.

Step	Riemann Hypothesis	P versus NP	Theorem
The native involution	The fold $s \mapsto 1 - \bar{s}$	Complementation $b \mapsto \neg b$	Theorem 19
The seat	The critical line, inhabited	Vacant, the ground of dimension zero	Theorem 19
The local fibre	Each prime's fibre is a copy of complementation	Complementation itself	Theorem 4

Step	Riemann Hypothesis	P versus NP	Theorem
The throne-holder	The primes, the bit on their side	Levin's universal search, the bit on its tail	Theorem 17
The record	Decides nothing	Decides nothing	Theorem 16
The form	Closed at theorem grade	Closed at theorem grade	Theorems 1 to 27
The value	Supplied by the act, at premise grade	Closed on the root read on computation, at premise grade	Theorems 6 and 8

10. The Frontier and the Route

Theorem 21 (The black box). A test-only generator correct over 2^N candidates tests every one of them (`black_box_bound`), on an axiom-free pigeonhole, and admits no savings (`black_box_no_savings`).

Theorem 22 (The reader). The black-box adversary is the point predicate, which a reader of its description solves in one step (`reader_solves`, `reader_poly`), so black-box bounds fail once descriptions reveal (`fails_when_descriptions_reveal`). The missing object is a lower bound against readers of the description, and it is keyed (`missing_object_is_keyed`).

Theorem 23 (The kinetic face). Every registration has a cost (`no_free_registration`); exhaustive search exhausts every budget past its candidates (`brute_force_exhausts`) while verification is cheaper (`asymmetry_chain`); and the energy gap is the operation gap in every frame (`frame_independent`), so the kinetic face, priced by Landauer's floor (Landauer 1961) with or without reversible steps (Bennett 1973), transports a gap and never produces one. The asymmetry, taken alone, decides nothing (`asymmetry_decides_nothing`).

Theorem 24 (The dynamics). Thermal relaxation is exponential in the barrier (`relaxation_exponential`), and on one satisfiability instance the barrier exists under single flips and vanishes under elimination moves (`barrier_belongs_to_the_dynamics`): a barrier is a property of the dynamics, not of the problem.

Theorem 25 (The prime bridge). Prime freedom carried onto the semiprime fibre leaves one orientation bit, walled and gauge (`orientation_wall`, `orientation_is_gauge`), with the witness determined by the description (`witness_determined_15`). Read as cost it gives a bridge sufficient for the separation and strictly stronger than it (`prime_bridge`, `prime_bridge_is_one_way`).

Theorem 26 (The route closes on the machine). Williams' theorem carries savings for a circuit class to $\text{NEXP} \not\subseteq$ that class (Williams 2011; Murray and Williams 2018), and the inclusion $P \subseteq P/\text{poly}$ carries $\text{NP} \not\subseteq P/\text{poly}$ to the machine's separation; both are carried as fields. The route reaches (`route_reaches`); supplied savings for general polynomial-size circuits and the scale-down from NEXP to NP , it closes the ma-

chine's own separation, on no axiom (`route_closes_on_the_machine`), and fills the closure itself (`the_route_fills_least_escape`). The relativization and natural-proof barriers are typed in the same kernel (`b1_no_relativizing_proof`, `b2_self_gag`), and the blocker of the prime bridge in three layers (`round2_terminal`).

Theorem 27 (The master seal). The closure gives the separation, the reading is the closure, the throne's holder decides the separation and is stopped at every bound on the reading, every prime's fibre is a copy of complementation, and the route closes the machine's separation on its two objects, bound in one theorem (`the_master_seal`).

11. Discussion

The arc. The paper runs from existence to the separation without a gap in its form. Existence and freedom are the ground, satisfiable and conservative; the deed instantiates existence at every act; the primes carry freedom and carry complementation in their fibres. The separation is defined on a machine and given by its closure on no axiom. The root read on computation is the separation, and it is the one place where existence meets it: every statement that reads the same on every machine, existence and freedom as written among them, is proved not to be the reading. The value closes on that reading.

The posit is exact. The root read on computation is one statement, equivalent to the separation, keyed, and the weakest premise on which the separation closes. Holding it is holding $P \neq \text{NP}$ at the grade of a posit, the standing of least erasure on the Riemann row, and every proof of $P \neq \text{NP}$ by any method fills it.

The next objects. Two, in order. The machine instantiated at Turing machines, with deciders taken total or run on fuel so that no looping machine counts as clean, so that the separation is the standard sentence and Levin's domination is compiled on it. The posit at theorem grade, which the route reduces to two typed objects, savings for general polynomial-size circuits and the scale-down from NEXP to NP .

Falsifiers. Three, each typed by the gate it strikes.

Prediction 1 (The computed refuter). A polynomial-time decider of satisfiability with every run clean refutes the reading on the actual machine,

by `rejection_is_a_witness`; it is the one refuter of the value.

Prediction 2 (The keyless refuter). A keyless premise shown to force the reading contradicts `keyless_forces_nothing` and `no_keyless_statement_is_least_escape`, compiled theorems, so it can arrive only as an error in Lean’s kernel.

Prediction 3 (The throne refuter). An algorithm that escapes while universal search does not refutes the throne’s field, Levin’s theorem.

How the paper stands to each prior position. The paper owes each position it engaged one statement of where it stands, in words with fixed senses. A result

is *additive* when it supplies a result or test the position lacked and leaves the position standing; *scoping* when it keeps a prior claim inside a stated boundary; *competing* when it argues a different route without executing a comparison; and *kin* when the paper is continuous with it. Superseding, which retires a theory’s central claim on executed data, is used nowhere. The census by row is eight additive, six scoping, one competing and four kin, with no replacing, subsuming, corroborating or contradicting row. The paper’s contribution to the literature is exactly the set of relations in Table 2, and nothing wider.

Table 2 | How the paper stands to each prior position. Relation words as defined in the paragraph above; superseding is used nowhere.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973; Karp 1972	Satisfiability is NP-complete, and the separation is posed for the classes they define.	Defines the separation on a machine deciding satisfiability within a polynomial bound.	kin	cited
Hartmanis and Stearns 1965	More time decides more languages.	Keeps the hierarchy’s separations to resource bounds of different order, by arguments that relativize.	scoping	cited
Baker, Gill and Solovay 1975	No relativizing argument settles the question.	Extends the two-world structure to every premise that reads the same on every machine (Theorems 11 and 13).	additive	executed
Razborov and Rudich 1997	No natural argument separates the classes if pseudorandom functions exist.	Types the barrier as a condition the posit at theorem grade must pass (Theorem 26).	kin	cited
Aaronson and Wigderson 2009	Algebrizing arguments cannot settle the question.	Names the barrier as a condition the posit at theorem grade must pass.	kin	cited
Razborov 1985	Monotone circuits for clique need superpolynomial size.	Reads the result as a special case of the posit, known in a restricted model.	scoping	cited
Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986	Parity has no constant-depth circuits of polynomial size.	Reads the results as special cases of the posit, known in a restricted model.	scoping	cited
Li and Yang 2022	An explicit function needs general circuits of size $3.1n - o(n)$.	Takes the bound as the distance between the frontier and the posit at theorem grade.	scoping	cited
Mulmuley and Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	Proposes a different route to the same object, without executing a comparison.	competing	cited
Cook and Reckhow 1979	$NP \neq coNP$ exactly when no proof system has polynomial-size proofs of all tautologies.	Stands beside it as a second conversion of the separation into one object, here proved equivalent by machine.	kin	cited
Deterministic query search	Black-box search over N candidates needs N tests.	Compiles the bound and shows a reader of the description beats it in one step (Theorems 21 and 22).	additive	executed
Bennett, Bernstein, Brassard and Vazirani 1997; Grover 1996	Quantum black-box search needs, and attains, order $\sqrt{N}$ queries.	Places the quantum bound beside Theorem 21, closed for test-only methods.	scoping	cited
Landauer 1961	Erasing one bit dissipates at least $k_B T \ln 2$.	Proves the energy gap is the operation gap in every frame, so the kinetic face transports a gap (Theorem 23).	additive	executed
Bennett 1973	Every computation can be run reversibly.	Proves the price question immaterial: the kinetic face transports a gap with or without reversible steps (Theorem 23).	additive	executed

Prior position	What it holds	What this paper does with it	Relation	Evidence
Islam 2026e	The heat of registration is zero exactly at the value on a chart with a seat.	Applies the seat to this problem: the seat vanishes, the ground of dimension zero (Theorem 19).	scoping	cited
Islam 2026a	Twenty-three open problems formalized from existence alone, the value excepted by theorem on every row.	Carries the P versus NP row to its closure on one posit (Theorems 6 to 12).	additive	executed
Islam 2026f	The structural verdict on P versus NP is the only one left standing.	Compiles the verdict on a machine: the vanishing seat, the record and the posit (Theorems 16, 19 and 9).	additive	executed
Levin 1973 (universal search)	One program runs every program at once and finds a satisfying assignment within a constant factor of the fastest; the decision form follows by self-reducibility.	Characterizes it as the holder of the machine's throne, deciding the separation on no axiom (Theorems 17 and 18).	additive	executed
Williams 2011; Murray and Williams 2018	Savings for satisfiability of a circuit class give NEXP outside that class.	Wires the route to the machine's own separation on its two objects (Theorem 26).	additive	executed

12. Conclusion

The P versus NP problem has been approached through diagonalization, circuit lower bounds, arithmetization, geometric complexity and proof complexity, and each stops at a theorem that limits its method. This paper anchors it in existence and closes it on one posit. Existence and freedom are the ground, satisfiable and conservative, carried by the primes, every prime's fibre a copy of complementation. The separation is defined on a machine and given by its closure on no axiom. The root read on computation is the separation, the one place where existence meets it, the single posit and proved irreducible, and $P \neq NP$ holds on it at premise grade.

Around the closure stand the double defense, the held throne, whose holder decides the separation on no axiom, the vanishing seat, and a frontier that types every route, with the open exit reduced to two named objects. The single open object is the reading at theorem grade, and every derivation of $P \neq NP$, by any method, fills it. All one hundred eighty theorems are compiled, pinned and printed for any reader to reproduce.

References

- Aaronson, S., and A. Wigderson. 2009. "Algebrization: A New Barrier in Complexity Theory." *ACM Transactions on Computation Theory* 1 (1): 2:1–2:54.
- Ajtai, M. 1983. " Σ^1_1 -Formulae on Finite Structures." *Annals of Pure and Applied Logic* 24 (1): 1–48.
- Baker, T., J. Gill, and R. Solovay. 1975. "Relativizations of the $P = ? NP$ Question." *SIAM Journal on Computing* 4 (4): 431–442.
- Bennett, C. H. 1973. "Logical Reversibility of Computation." *IBM Journal of Research and Development* 17 (6): 525–532.

Bennett, C. H., E. Bernstein, G. Brassard, and U. Vazirani. 1997. "Strengths and Weaknesses of Quantum Computing." *SIAM Journal on Computing* 26 (5): 1510–1523.

Cook, S. A. 1971. "The Complexity of Theorem-Proving Procedures." In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.

Cook, S. A., and R. A. Reckhow. 1979. "The Relative Efficiency of Propositional Proof Systems." *Journal of Symbolic Logic* 44 (1): 36–50.

de Moura, L., and S. Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." In *Automated Deduction, CADE 28*, 625–635.

Furst, M., J. B. Saxe, and M. Sipser. 1984. "Parity, Circuits, and the Polynomial-Time Hierarchy." *Mathematical Systems Theory* 17 (1): 13–27.

Grover, L. K. 1996. "A Fast Quantum Mechanical Algorithm for Database Search." In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 212–219.

Hartmanis, J., and R. E. Stearns. 1965. "On the Computational Complexity of Algorithms." *Transactions of the American Mathematical Society* 117: 285–306.

Håstad, J. 1986. "Almost Optimal Lower Bounds for Small Depth Circuits." In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, 6–20.

Islam, M. F. 2026a. "A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, the Value Excepted by Theorem on Every Row." Zenodo. <https://doi.org/10.5281/zenodo.22913635>.

Islam, M. F. 2026b. "Trisduction: The Master Codex 5.0.0, Built from the Root." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117441>.

Islam, M. F. 2026c. "PhysOST^T: The Trisduction Physical Operating System, Public Edition 1.0.9p." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117440>.

Islam, M. F. 2026d. “Trisduction: The Geometric Mother Codex, the Unabridged Register of Record, Edition 3.46.0.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117445>.

Islam, M. F. 2026e. “The Bridge From First Principle, Edition 1.5.0, with The Final Cut.” Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23105348>.

Islam, M. F. 2026f. “Why the Structural Verdict on P versus NP Is the Only One Left Standing: Mathematics Proves That Mathematics Cannot Rule Here.” Zenodo. <https://doi.org/10.5281/zenodo.21368936>.

Karp, R. M. 1972. “Reducibility among Combinatorial Problems.” In *Complexity of Computer Computations*, edited by R. E. Miller and J. W. Thatcher, 85–103. New York: Plenum.

Landauer, R. 1961. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development* 5 (3): 183–191.

Levin, L. A. 1973. “Universal Sequential Search Problems.” *Problemy Peredachi Informatsii* 9 (3): 115–116.

Li, J., and T. Yang. 2022. “ $3.1n - o(n)$ Circuit Lower Bounds for Explicit Functions.” In *Proceedings of the 54th Annual ACM Symposium on Theory of Computing*, 1180–1193.

Mulmuley, K. D., and M. Sohoni. 2001. “Geometric Complexity Theory I: An Approach to the P vs. NP and Related Problems.” *SIAM Journal on Computing* 31 (2): 496–526.

Murray, C., and R. Williams. 2018. “Circuit Lower Bounds for Nondeterministic Quasi-Polytime: An Easy Witness Lemma for NP and NQP.” In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing*, 890–901.

Razborov, A. A. 1985. “Lower Bounds on the Monotone Complexity of Some Boolean Functions.” *Soviet Mathematics Doklady* 31: 354–357.

Razborov, A. A., and S. Rudich. 1997. “Natural Proofs.” *Journal of Computer and System Sciences* 55 (1): 24–35.

Williams, R. 2011. “Non-uniform ACC Circuit Lower Bounds.” In *Proceedings of the 26th IEEE Conference on Computational Complexity*, 115–125.

Appendix A. The Fortification Kernel, PNP_Fortification.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the machine, the throne, the reading, the prime’s triaxis and the route, twenty theorems. SHA-256: d6b155783e3185cca9b9b367e520ce13ba8ac0ee52f691a7890cc7db0bbc13b3

```
1 /-
2 PNP_Fortification.lean · APEX-PSP-PNP-SEED-03 · The fortification of least escape,
3 on SEED-02's own machine
4 I · the machine, restated verbatim from PNP_Least_Escape.lean, so that every
5 theorem here is about the same
```

```
4 separation. II · the throne over the machine: a holder escapes whenever any
5 algorithm escapes, and then the
6 separation holds exactly when the holder never escapes; Levin's universal search
7 is the holder of the
8 satisfiability machine by Levin's theorem (1973), carried as the throne's field.
9 III · the two posits, the bridge
10 of SEED-01 and least escape of SEED-02, proved one. IV · the prime's triaxis:
11 every prime's fibre is exactly two
12 pairs, lies off the multiplicative seat, and is a copy of complementation. V · the
13 master seal. Core Lean 4, no
14 import, no axiom declared, no sorry.
15
16 /-
17 namespace PNP.FORTIFY
18
19 /-! ## I · The machine, restated verbatim -/
20
21 structure Machine where
22   Alg : Type
23   Inst : Type
24   size : Inst → Nat
25   out : Alg → Inst → Bool
26   time : Alg → Inst → Nat
27   sat : Inst → Bool
28
29 def bound (c k n : Nat) : Nat := c * n ^ k + c
30
31 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
32   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
33
34 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
35
36 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
37
38 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
39
40 structure LeastEscape (M : Machine) where
41   closure : NothingEscapes M
42
43 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
44   fun (A, c, k, he) => match h A c k with
45   | (x, hx) => hx (he x)
46
47 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
48   fun A c k => Classical.byContradiction fun hn =>
49     h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
50
51 /-! ## II · The throne over the machine -/
52
53 /- A THRONE of the machine: a holder that escapes whenever any algorithm escapes.
54 For the satisfiability machine,
55 Levin's universal search is a holder by Levin's theorem (1973), carried here as
56 the field. -/
57
58 structure Throne (M : Machine) where
59   holder : M.Alg
60   dominates : ∀ A c k, Escapes M A c k → ∃ c' k', Escapes M holder c' k'
61
62 /- THE HOLDER DECIDES THE SEPARATION: the separation holds exactly when the holder
63 never escapes. -/
64
65 theorem the_holder_decides_the_separation (M : Machine) (T : Throne M) :
66   Sep M ↔ ∃ c k, Escapes M T holder c k :=
67   (fun hs (c, k, he) => hs (T holder, c, k, he),
68    fun hn (A, c, k, he) => hn (T.dominates A c k he))
69
70 /- THE HOLDER IS UNIQUE: any two holders escape together or not at all. -/
71
72 theorem the_holder_is_unique (M : Machine) (T T' : Throne M) :
73   (∃ c k, Escapes M T holder c k) ↔ (∃ c k, Escapes M T' holder c k) :=
74   (fun (c, k, he) => T'.dominates T holder c k he, fun (c, k, he) => T.dominates
75     T' holder c k he)
76
77 /- LEAST ESCAPE AT THE THRONE: nothing escapes exactly when the holder never
78 escapes. -/
79
80 theorem least_escape_at_the_throne (M : Machine) (T : Throne M) :
81   NothingEscapes M ↔ ∃ c k, Escapes M T holder c k :=
82   (fun h => (the_holder_decides_the_separation M T).mp (sep_of_closure M h),
83    fun h => closure_of_sep M ((the_holder_decides_the_separation M T).mpr h))
84
85 /- THE ACT AT THE THRONE: on least escape, the holder meets, at every bound, an
86 instance that stops it. -/
87
88 theorem the_act_stops_the_holder (M : Machine) (T : Throne M) (L : LeastEscape M) :
89   ∀ c k, ∃ x, ¬ Clean M T holder c k x :=
90   fun c k => L.closure T holder c k
91
92 /-! ## III · The two posits are one -/
93
94 structure Point where
95   offset : Int
96   height : Nat
97   deriving DecidableEq
98
99 def reg (p : Point) : Point := (0, p.height)
100
101 def side (p : Point) : Bool := decide (0 < p.offset)
102
103 theorem destroyed_is_unrecoverable : ¬ ∃ g : Point → Bool, ∀ p, g (reg p) = side p
104 := by
105   intro (g, hg)
106   have h1 := hg (1, 0); have h2 := hg (-1, 0)
107   simp [reg, side] at h1 h2
108   rw [h1] at h2; exact Bool.noConfusion h2
109
110 theorem hidden_is_recoverable {α : Type} (xs : List α) (hxs : ∀ x, x ∈ xs) (x₀ : α) :
111   ∃ g : (α → Bool) → α, ∀ f : α → Bool, (∃ x, f x = true) → f (g f) = true := by
112   refine (fun f => (xs.find? f).getD x₀, fun f (x, hx) => ?_)
113   cases h : xs.find? f with
114   | some y => simp [h] using List.find?_some h
115   | none => exact absurd hx (by simp using List.find?_eq_none.mp h x (hxs x))
116
117 def Asymmetry : Prop :=
118   (¬ ∃ g : Point → Bool, ∀ p, g (reg p) = side p) ∧
119   (∃ g : (Bool → Bool) → Bool, ∀ f : Bool → Bool, (∃ x, f x = true) → f (g f) =
120     true)
```

```

97 theorem asymmetry_holds : Asymmetry :=
98   (destroyed_is_unrecoverable, hidden_is_recoverable [true, false] (fun x => by
99     cases x <=> simp) true)
100
101 /-- THE TWO POSITS ARE ONE: SEED-01's bridge, read on the machine, is exactly
102 SEED-02's least escape. -/
103 theorem the_two_posits_are_one (M : Machine) : (Asymmetry → Sep M) → NothingEscapes
104 M :=
105   (fun b => closure_of_sep M (b.mp asymmetry_holds), fun h => (fun _ =>
106     sep_of_closure M h, fun _ => asymmetry_holds))
107
108 /-- EVERY PREMISE THAT CLOSES THE SEPARATION FILLS BOTH POSITS. -/
109 theorem every_closing_premise_fills_both (M : Machine) (Q : Prop) (h : Q → Sep M) :
110   Q → (Asymmetry → Sep M) ∧ NothingEscapes M :=
111   fun q => ((fun _ => h q, fun _ => asymmetry_holds), closure_of_sep M (h q))
112
113 /-! ## IV · The prime's triaxis -/
114
115 def IsPrime (n : Nat) : Prop := 2 ≤ n ∧ ∀ m, m | n → m = 1 ∨ m = n
116
117 /-- FORM: the fibre of multiplication over a prime is exactly the two trivial pairs. -/
118 theorem prime_fibre (p : Nat) (hp : IsPrime p) (a b : Nat) :
119   a * b = p ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1) := by
120   constructor
121   · intro h
122     rcases hp.2 a (b, h.symm) with ha | ha
123     · left; refine (ha, ?_); rw [ha, Nat.one_mul] at h; exact h
124     · right; refine (ha, ?_)
125       rw [ha] at h
126       have h2 := hp.1
127       by_cases hb0 : b = 0
128       · rw [hb0, Nat.mul_zero] at h; omega
129       · by_cases hb1 : b = 1
130       · exact hb1
131       · have hb2 : 2 ≤ b := by omega
132       have : p = 2 ≤ p * b := Nat.mul_le_mul_left p hb2
133       omega
134   · rintro ((rfl, rfl) | (rfl, rfl))
135   · exact Nat.one_mul _
136   · exact Nat.mul_one _
137
138 /-- SEAT: a prime lies off the multiplicative seat. -/
139 theorem prime_off_seat (p : Nat) (hp : IsPrime p) : ~ ∃ a, a * a = p := by
140   intro a, ha
141   have h2 := hp.1
142   rcases (prime_fibre p hp a a).mp ha with (h1, h3) | (h1, h3) <=> omega
143
144 def Fibre (n : Nat) := {x : Nat × Nat // x.1 * x.2 = n}
145 def swapF {n : Nat} (x : Fibre n) : Fibre n := {(x.1.2, x.1.1), by rw
146   [Nat.mul_comm]; exact x.2}
147 def toBit {n : Nat} (x : Fibre n) : Bool := decide (x.1.1 = 1)
148
149 /-- THE BIT: every prime's fibre with its swap is the decision bit with
150 complementation. -/
151 theorem prime_fibre_is_complementation (p : Nat) (hp : IsPrime p) :
152   (∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
153   (∀ x y : Fibre p, toBit x = toBit y → x = y) ∧ (∀ b : Bool, ∃ x : Fibre p, toBit
154     x = b) := by
155   have hp1 : p ≠ 1 := by have : hp.1; omega
156   refine (fun x => ?_, fun x y hxy => ?_, fun b => ?_)
157   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (h1, h2) | (h1, h2)
158   · simp [toBit, swapF, h1, h2, hp1]
159   · simp [toBit, swapF, h1, h2, hp1]
160   · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (a1, a2) | (a1, a2) <=>
161     rcases (prime_fibre p hp y.1.1 y.1.2).mp y.2 with (b1, b2) | (b1, b2)
162     · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans b2.symm))
163     · simp [toBit, a1, b1, hp1] at hxy
164     · simp [toBit, a1, b1, hp1] at hxy
165     · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans b2.symm))
166   · cases b
167   · exact (((p, 1), Nat.mul_one p), by simp [toBit, hp1])
168   · exact (((1, p), Nat.one_mul p), by simp [toBit])
169
170 /-! ## VI · The three names are one -/
171
172 /-- RA READ ON COMPUTATION: every decider of satisfiability actuates beyond every
173 polynomial bound. -/
174 def RAcop (M : Machine) : Prop := ∀ A c k, ~ Escapes M A c k
175
176 theorem ra_comp_is_sep (M : Machine) : RAcop M → Sep M :=
177   (fun h (A, c, k, he) => h A c k he, fun h A c k he => h (A, c, k, he))
178
179 /-- THE THREE NAMES ARE ONE: the bridge, least escape, and RA read on computation
180 are one posit, each equivalent to
181 the separation, on one machine. -/
182 theorem the_three_names_are_one (M : Machine) :
183   ((Asymmetry → Sep M) → NothingEscapes M) ∧ (NothingEscapes M → RAcop M) ∧
184   (RAcop M → Sep M) :=
185   (the_two_posits_are_one M,
186     (fun h => (ra_comp_is_sep M).mpr (sep_of_closure M h), fun h => closure_of_sep M
187       (ra_comp_is_sep M).mp h),
188     ra_comp_is_sep M)
189
190 /-! ## VII · The Williams route, wired to the machine -/
191
192 /-- THE ROUTE ON THE MACHINE. 'Savings C' is a white-box satisfiability algorithm
193 for circuit class C beating 2^n by
194 a superpolynomial factor; 'williams' is Williams' theorem, savings for C give
195 NEXP ≠ C, carried; 'Ppoly' is the
196 class of polynomial-size circuits; 'np_not_ppoly_gives_sep' is the elementary
197 inclusion P ≤ P/poly, carried, by
198 which NP ≠ P/poly gives the machine's own separation. -/
199 structure Route (M : Machine) where
200   Class      : Type
201   Savings    : Class → Prop

```

```

199 NEXPnotIn : Class → Prop
200 williams   : ∀ C, Savings C → NEXPnotIn C
201 Ppoly      : Class
202 NNotPpoly  : Prop
203 np_not_ppoly_gives_sep : NNotPpoly → Sep M
204
205 /-- THE ROUTE REACHES: savings for a class give NEXP ≠ that class. -/
206 theorem route_reaches (M : Machine) (R : Route M) (C : R.Class) (s : R.Savings C) :
207   R.NEXPnotIn C :=
208   R.williams C s
209
210 /-- THE ROUTE CLOSES ON THE MACHINE: savings for general circuits and the scale-down
211 from NEXP to NP give the
212 machine's own separation. -/
213 theorem route_closes_on_the_machine (M : Machine) (R : Route M) (s : R.Savings
214   R.Ppoly)
215   (scaleDown : R.NEXPnotIn R.Ppoly → R.NNotPpoly) : Sep M :=
216   R.np_not_ppoly_gives_sep (scaleDown (R.williams R.Ppoly s))
217
218 /-- THE ROUTE FILLS THE POSIT: supplied its two objects, the route fills least
219 escape itself. -/
220 theorem the_route_fills_least_escape (M : Machine) (R : Route M) (s : R.Savings
221   R.Ppoly)
222   (scaleDown : R.NEXPnotIn R.Ppoly → R.NNotPpoly) : LeastEscape M :=
223   (closure_of_sep M (route_closes_on_the_machine M R s scaleDown))
224
225 /-! ## V · The master seal -/
226
227 /-- THE MASTER SEAL: least escape closes the separation; the two posits are one, and
228 the third name, RA read on
229 computation, with them; the throne's holder decides the separation; on least
230 escape the holder is stopped at
231 every bound; every prime's fibre is a copy of complementation; and the Williams
232 route, supplied its two objects,
233 closes the machine's own separation. -/
234 theorem the_master_seal (M : Machine) (T : Throne M) :
235   (NothingEscapes M → Sep M) ∧ ((Asymmetry → Sep M) → NothingEscapes M) ∧
236   (Sep M → ~ ∃ c k, Escapes M T.holder c k) ∧
237   (∀ _ : LeastEscape M, ∀ c k, ∃ x, ~ Clean M T.holder c k x) ∧
238   (∀ p, IsPrime p → ∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
239   (NothingEscapes M → RAcop M) ∧
240   (∀ R : Route M, R.Savings R.Ppoly → (R.NEXPnotIn R.Ppoly → R.NNotPpoly) → Sep
241     M) :=
242   (sep_of_closure M, the_two_posits_are_one M, the_holder_decides_the_separation M
243     T,
244     fun L => the_act_stops_the_holder M T L, fun p hp =>
245       (prime_fibre_is_complementation p hp).1,
246       (the_three_names_are_one M).2.1, fun R s d => route_closes_on_the_machine M R s
247         d)
248
249 end PNP.FORTIFY
250
251 /-! ## Cones, pinned as printed -/
252 /-- info: 'PNP.FORTIFY.sep_of_closure' does not depend on any axioms -/
253 #guard_msgs in #print axioms PNP.FORTIFY.sep_of_closure
254 /-- info: 'PNP.FORTIFY.closure_of_sep' depends on axioms: [propext,
255   Classical.choice, Quot.sound] -/
256 #guard_msgs in #print axioms PNP.FORTIFY.closure_of_sep
257 /-- info: 'PNP.FORTIFY.the_holder_decides_the_separation' does not depend on any
258   axioms -/
259 #guard_msgs in #print axioms PNP.FORTIFY.the_holder_decides_the_separation
260 /-- info: 'PNP.FORTIFY.the_holder_is_unique' does not depend on any axioms -/
261 #guard_msgs in #print axioms PNP.FORTIFY.the_holder_is_unique
262 /-- info: 'PNP.FORTIFY.least_escape_at_the_throne' depends on axioms: [propext,
263   Classical.choice, Quot.sound] -/
264 #guard_msgs in #print axioms PNP.FORTIFY.least_escape_at_the_throne
265 /-- info: 'PNP.FORTIFY.the_act_stops_the_holder' does not depend on any axioms -/
266 #guard_msgs in #print axioms PNP.FORTIFY.the_act_stops_the_holder
267 /-- info: 'PNP.FORTIFY.destroyed_is_unrecoverable' depends on axioms: [propext] -/
268 #guard_msgs in #print axioms PNP.FORTIFY.destroyed_is_unrecoverable
269 /-- info: 'PNP.FORTIFY.hidden_is_recoverable' depends on axioms: [propext,
270   Quot.sound] -/
271 #guard_msgs in #print axioms PNP.FORTIFY.hidden_is_recoverable
272 /-- info: 'PNP.FORTIFY.asymmetry_holds' depends on axioms: [propext, Quot.sound] -/
273 #guard_msgs in #print axioms PNP.FORTIFY.asymmetry_holds
274 /-- info: 'PNP.FORTIFY.the_two_posits_are_one' depends on axioms: [propext,
275   Classical.choice, Quot.sound] -/
276 #guard_msgs in #print axioms PNP.FORTIFY.the_two_posits_are_one
277 /-- info: 'PNP.FORTIFY.every_closing_premise_fills_both' depends on axioms:
278   [propext, Classical.choice, Quot.sound] -/
279 #guard_msgs in #print axioms PNP.FORTIFY.every_closing_premise_fills_both
280 /-- info: 'PNP.FORTIFY.prime_fibre' depends on axioms: [propext, Quot.sound] -/
281 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre
282 /-- info: 'PNP.FORTIFY.prime_off_seat' depends on axioms: [propext, Quot.sound] -/
283 #guard_msgs in #print axioms PNP.FORTIFY.prime_off_seat
284 /-- info: 'PNP.FORTIFY.prime_fibre_is_complementation' depends on axioms: [propext,
285   Quot.sound] -/
286 #guard_msgs in #print axioms PNP.FORTIFY.prime_fibre_is_complementation
287 /-- info: 'PNP.FORTIFY.ra_comp_is_sep' does not depend on any axioms -/
288 #guard_msgs in #print axioms PNP.FORTIFY.ra_comp_is_sep
289 /-- info: 'PNP.FORTIFY.the_three_names_are_one' depends on axioms: [propext,
290   Classical.choice, Quot.sound] -/
291 #guard_msgs in #print axioms PNP.FORTIFY.the_three_names_are_one
292 /-- info: 'PNP.FORTIFY.route_reaches' does not depend on any axioms -/
293 #guard_msgs in #print axioms PNP.FORTIFY.route_reaches
294 /-- info: 'PNP.FORTIFY.route_closes_on_the_machine' does not depend on any axioms -/
295 #guard_msgs in #print axioms PNP.FORTIFY.route_closes_on_the_machine
296 /-- info: 'PNP.FORTIFY.the_route_fills_least_escape' depends on axioms: [propext,
297   Classical.choice, Quot.sound] -/
298 #guard_msgs in #print axioms PNP.FORTIFY.the_route_fills_least_escape
299 /-- info: 'PNP.FORTIFY.the_master_seal' depends on axioms: [propext,
300   Classical.choice, Quot.sound] -/
301 #guard_msgs in #print axioms PNP.FORTIFY.the_master_seal

```

Appendix B. The Closure Kernel, PNP_Least_Escape.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the closure and its double defense, twenty-four theorems. SHA-256: cfd2d9d6ea7d4f7b03ae07dd5976539d1168ff3da3c9d024f3b19518044f33be

```

1  /-!
2  # PNP_Least_Escape.lean · APEX-PSP-PNP-SEED-02 · Least Escape, the seal and its
3  double defense, hardened: Omega and AEGIS of the row
4
5  The P versus NP twin of PSP-RH-SEAL-01's `double_defense_hardened`.
6
7  OMEGA, the formal gate. Nothing passes for the separation but least escape. The
8  socket of the act is the closure; every premise that forces the separation fills
9  the field of the act; every premise either forces the closure or has a twin
10 machine
11 on which it holds and the closure fails; a keyless premise forces nothing.
12
13 AEGIS, the actuation gate. Nothing passes against least escape but a computed
14 escape: an algorithm, a polynomial bound, and every run clean. A finite stock of
15 examinations decides nothing. The root, which every computation re-enacts, crosses
16 neither way. The refusal is the same under every logic.
17
18 THE COSMIC READING, at its exact scope. Every pair a substrate can examine lands
19 on
20 exactly one gate: a stopping instance, which confirms least escape at that pair,
21 or
22 a clean escape, which refutes it. There is no third landing. Every confirming
23 computation, anywhere, is an atomic witness of least escape at its pair and pays
24 the
25 floor; under the act, every pair confirms and no computation escapes. What no
26 computation does is enact the universal: the universal is the act.
27
28 ARMOR SCOPE. The defense forces the form of every objection and never the value:
29 the closure holds on one coherent machine and fails on another. Without the act
30 the
31 value is marked [.].
32
33 Core Lean 4, no import. Cones pinned at the foot, each as printed by the kernel.
34 -/
35
36 namespace PNP.HARDENED
37
38 /-! ## 0 · the machine, the closure, the act -/
39 structure Machine where
40   Alg : Type
41   Inst : Type
42   size : Inst → Nat
43   out : Alg → Inst → Bool
44   time : Alg → Inst → Nat
45   sat : Inst → Bool
46
47 def bound (c k n : Nat) : Nat := c * n ^ k + c
48 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
49   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
50 def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M A c k x
51 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
52 def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
53
54 structure LeastEscape (M : Machine) where
55   closure : NothingEscapes M
56
57 structure Examination (M : Machine) where
58   A : M.Alg
59   c : Nat
60   k : Nat
61   x : M.Inst
62   stops : ¬ Clean M A c k x
63
64 theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
65   fun (A, c, k, he) => match h A c k with
66   | (x, hx) => hx (he x)
67
68 theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
69   fun A c k => Classical.byContradiction fun hn =>
70     h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
71
72 theorem sep_from_the_act (M : Machine) (L : LeastEscape M) : Sep M := sep_of_closure
73   M L.closure
74
75 /-! ## the two coherent machines -/
76 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
77   | 0 => rfl
78   | n + 1 => beq_self n
79
80 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
81   cases hb : Nat.beq x K with
82   | false => rfl
83   | true =>
84     have e : x = K := Nat.eq_of_beq_eq_true hb
85     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
86
87 def stageWorld (n : Nat) : Machine :=
88   (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ _ => 0, fun _ => true)
89 def closedWorld : Machine :=
90   (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ => true)
91
92 theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
93   fun _ => (beq_self n, Nat.zero_le _)
94 theorem closed_world_closes : NothingEscapes closedWorld :=

```

```

88   fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
89 theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld n) :=
90   fun h => sep_of_closure _ h (n, 0, 0, stage_escapes n)
91
92 /-! ## I · OMEGA, the formal gate -/
93
94 /- The socket of the act is the closure. -/
95 theorem socket_is_the_value (M : Machine) : Nonempty (LeastEscape M) →
96   NothingEscapes M :=
97   (fun (L) => L.closure, fun h => {(h)})
98
99 /- Every premise that forces the separation fills the field of the act. -/
100 theorem forcing_premise_fills_the_act (M : Machine) (Q : Prop) (h : Q → Sep M) :
101   Q → Nonempty (LeastEscape M) :=
102   fun q => {(closure_of_sep M (h q))}
103
104 def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M
105
106 /- Every premise forces the closure or has a twin machine where it holds and the
107 closure fails. -/
108 theorem forces_or_has_a_twin (P : Machine → Prop) :
109   Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
110   Classical.byCases (fun h : Forces P => Or.inl h) (fun h => Or.inr
111     (Classical.byContradiction fun hn => h fun M hP =>
112       Classical.byContradiction fun hc => hn (M, hP, hc)))
113
114 def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
115
116 /- A keyless premise forces nothing. -/
117 theorem keyless_forces_nothing (P : Machine → Prop) (hP : Keyless P) : ¬ Forces P :=
118   fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))
119
120 /-! ## II · AEGIS, the actuation gate -/
121
122 /- A computed escape refutes the closure. -/
123 theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (h : Escapes M A c k) :
124   ¬ NothingEscapes M :=
125   fun hc => sep_of_closure M hc (A, c, k, h)
126
127 /- Nothing passes against the closure but a computed escape. -/
128 theorem rejection_is_a_witness (M : Machine) :
129   ¬ NothingEscapes M → ∃ A c k, Escapes M A c k :=
130   (fun h => Classical.byContradiction fun hn => h (closure_of_sep M hn),
131     fun (A, c, k, he) => escape_refutes M A c k he)
132
133 /- A finite stock of examinations decides nothing. -/
134 theorem record_decides_nothing (n : Nat) :
135   (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ()) ∧ ¬ NothingEscapes
136     (stageWorld n) :=
137   (fun A _ _ hA (h, _) => by
138     have h' : Nat.beq A n = true := h
139     rw [beq_false_of_lt A n hA] at h'
140     exact Bool.noConfusion h',
141     stage_fails_closure n)
142
143 /- The root, which every computation re-enacts, crosses neither way. -/
144 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
145
146 theorem root_crosses_neither_way :
147   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → NothingEscapes
148     M) ∧
149   (¬ ∀ M : Machine, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
150     NothingEscapes M) :=
151   (fun h => stage_fails_closure 0 (h (stageWorld 0) (fun _ _ => trivial))),
152   fun h => h closedWorld (fun _ _ => trivial) closed_world_closes)
153
154 /- The refusal is constant under every logic. -/
155 def refusal (_logic : String) : String :=
156   "nothing passes against least escape but a computed escape"
157
158 theorem aegis_constant (l1 l2 : String) : refusal l1 = refusal l2 := rfl
159
160 /-! ## III · the cosmic reading, at its exact scope -/
161
162 /- Every examinable pair lands on exactly one gate: a stopping instance or a clean
163 escape. -/
164 theorem every_pair_lands (M : Machine) (A : M.Alg) (c k : Nat) :
165   ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) ∧
166   ¬ ((∃ x, ¬ Clean M A c k x) ∧ Escapes M A c k) :=
167   (Classical.byCases (fun h : Escapes M A c k => Or.inr h)
168     (fun h => Or.inl (Classical.byContradiction fun hn =>
169       h fun x => Classical.byContradiction fun hx => hn (x, hx))),
170     fun ({x, hx}, he) => hx (he x))
171
172 /- A confirming computation is an atomic witness of least escape at its pair. -/
173 theorem confirmation_is_atomic (M : Machine) (e : Examination M) :
174   (∃ x, ¬ Clean M e.A e.c e.k x) ∧ ¬ Escapes M e.A e.c e.k :=
175   ((e.x, e.stops), fun he => e.stops (he e.x))
176
177 /- Every confirming computation pays the floor. -/
178 def examCost (floor : Nat) (_e : Examination M) : Nat := floor * 1
179
180 theorem confirmation_pays (M : Machine) (floor : Nat) (h : 0 < floor) (e :
181   Examination M) :
182   0 < examCost floor e :=
183   Nat.mul_pos h (Nat.zero_lt_one)
184
185 /- Under the act, every pair confirms and no computation escapes. -/
186 theorem act_leaves_no_escape (M : Machine) (L : LeastEscape M) :
187   (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) ∧
188   (∀ A c k, ¬ Escapes M A c k) :=
189   (fun A c k => match L.closure A c k with
190   | (x, hx) => {(A, c, k, x, hx), rfl, rfl, rfl},
191     fun A c k he => sep_from_the_act M L (A, c, k, he))
192
193 /-! ## IV · armor scope and silence -/
194 theorem armor_scope : NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0)
195 :=

```

```

188 (closed_world_closes, stage_fails_closure 0)
189
190 inductive Token : Type
191 | sealSep | dot
192 deriving DecidableEq, Repr
193
194 /-- The emission: with the act supplied, the seal; without it, the dot. -/
195 def verdict (actSupplied : Bool) : Token := if actSupplied then .sealSep else .dot
196
197 theorem value_marked_dot : verdict false = .dot ∧ verdict true = .sealSep := (rfl,
198   rfl)
199
200 /-! ## V · THE DOUBLE DEFENSE, HARDENED -/
201 theorem double_defense_harden :
202   -- the act carries the separation
203   (V (M : Machine), LeastEscape M → Sep M) ∧
204   -- OMEGA
205   (V M : Machine, Nonempty (LeastEscape M) → NothingEscapes M) ∧
206   (V M : Machine) (Q : Prop), (Q → Sep M) → Q → Nonempty (LeastEscape M)) ∧
207   (V P : Machine → Prop, Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M) ∧
208   (V P : Machine → Prop, Keyless P → ¬ Forces P) ∧
209   -- AEGIS
210   (V M : Machine, ¬ NothingEscapes M → ∃ A c k, Escapes M A c k) ∧
211   (V n : Nat, (V A c k, A < n → ¬ Clean (stageWorld n) A c k ())) ∧
212   (¬ NothingEscapes (stageWorld n)) ∧
213   ((¬ V M : Machine, (V (U : Type) (ΔE : U → Int), RA U ΔE → True) →
214     NothingEscapes M) ∧
215     (¬ V M : Machine, (V (U : Type) (ΔE : U → Int), RA U ΔE → True) → ¬
216       NothingEscapes M)) ∧
217   (V l₁ l₂ : String, refusal l₁ = refusal l₂) ∧
218   -- the cosmic reading
219   (V (M : Machine) (A : M.Alg) (c k : Nat),
220     ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) ∧ ¬ ((∃ x, ¬ Clean M A c k x) ∧
221       Escapes M A c k)) ∧
222   (V (M : Machine) (e : Examination M), (∃ x, ¬ Clean M e A c e k x) ∧ ¬ Escapes
223     M e A c e k) ∧
224   (V (M : Machine), LeastEscape M →
225     (V A c k, ∃ e : Examination M, e A = A A e c = c A e k = k) ∧ (V A c k, ¬
226       Escapes M A c k)) ∧
227   -- armor scope and silence
228   (NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0)) ∧
229   (verdict false = .dot ∧ verdict true = .sealSep) :=
230   (sep_from_the_act, socket_is_the_value, forcing_premise_fills_the_act,
231     forces_or_has_a_twin,
232     keyless_forces_nothing, rejection_is_a_witness, record_decides_nothing,
233     root_crosses_neither_way,
234     aegis_constant, every_pair_lands, confirmation_is_atomic, act_leaves_no_escape,
235     armor_scope,
236     value_marked_dot)
237
238 end PNP.HARDENED
239
240 /-! ## Cones, pinned as printed -/
241 /-- info: 'PNP.HARDENED.sep_of_closure' does not depend on any axioms -/
242 #guard_msgs in #print axioms PNP.HARDENED.sep_of_closure
243 /-- info: 'PNP.HARDENED.closure_of_sep' depends on axioms: [propext,
244   Classical.choice, Quot.sound] -/
245 #guard_msgs in #print axioms PNP.HARDENED.closure_of_sep
246 /-- info: 'PNP.HARDENED.sep_from_the_act' does not depend on any axioms -/
247 #guard_msgs in #print axioms PNP.HARDENED.sep_from_the_act
248 /-- info: 'PNP.HARDENED.beq_self' does not depend on any axioms -/
249 #guard_msgs in #print axioms PNP.HARDENED.beq_self
250 /-- info: 'PNP.HARDENED.beq_false_of_lt' does not depend on any axioms -/
251 #guard_msgs in #print axioms PNP.HARDENED.beq_false_of_lt
252 /-- info: 'PNP.HARDENED.stage_escapes' does not depend on any axioms -/
253 #guard_msgs in #print axioms PNP.HARDENED.stage_escapes
254 /-- info: 'PNP.HARDENED.closed_world_closes' does not depend on any axioms -/
255 #guard_msgs in #print axioms PNP.HARDENED.closed_world_closes
256 /-- info: 'PNP.HARDENED.stage_fails_closure' does not depend on any axioms -/
257 #guard_msgs in #print axioms PNP.HARDENED.stage_fails_closure
258 /-- info: 'PNP.HARDENED.socket_is_the_value' does not depend on any axioms -/
259 #guard_msgs in #print axioms PNP.HARDENED.socket_is_the_value
260 /-- info: 'PNP.HARDENED.forcing_premise_fills_the_act' depends on axioms: [propext,
261   Classical.choice, Quot.sound] -/
262 #guard_msgs in #print axioms PNP.HARDENED.forcing_premise_fills_the_act
263 /-- info: 'PNP.HARDENED.forces_or_has_a_twin' depends on axioms: [propext,
264   Classical.choice, Quot.sound] -/
265 #guard_msgs in #print axioms PNP.HARDENED.forces_or_has_a_twin
266 /-- info: 'PNP.HARDENED.keyless_forces_nothing' does not depend on any axioms -/
267 #guard_msgs in #print axioms PNP.HARDENED.keyless_forces_nothing
268 /-- info: 'PNP.HARDENED.escape_refutes' does not depend on any axioms -/
269 #guard_msgs in #print axioms PNP.HARDENED.escape_refutes
270 /-- info: 'PNP.HARDENED.rejection_is_a_witness' depends on axioms: [propext,
271   Classical.choice, Quot.sound] -/
272 #guard_msgs in #print axioms PNP.HARDENED.rejection_is_a_witness
273 /-- info: 'PNP.HARDENED.record_decides_nothing' does not depend on any axioms -/
274 #guard_msgs in #print axioms PNP.HARDENED.record_decides_nothing
275 /-- info: 'PNP.HARDENED.root_crosses_neither_way' does not depend on any axioms -/
276 #guard_msgs in #print axioms PNP.HARDENED.root_crosses_neither_way
277 /-- info: 'PNP.HARDENED.aegis_constant' does not depend on any axioms -/
278 #guard_msgs in #print axioms PNP.HARDENED.aegis_constant
279 /-- info: 'PNP.HARDENED.every_pair_lands' depends on axioms: [propext,
280   Classical.choice, Quot.sound] -/
281 #guard_msgs in #print axioms PNP.HARDENED.every_pair_lands
282 /-- info: 'PNP.HARDENED.confirmation_is_atomic' does not depend on any axioms -/
283 #guard_msgs in #print axioms PNP.HARDENED.confirmation_is_atomic
284 /-- info: 'PNP.HARDENED.confirmation_pays' does not depend on any axioms -/
285 #guard_msgs in #print axioms PNP.HARDENED.confirmation_pays
286 /-- info: 'PNP.HARDENED.act_leaves_no_escape' does not depend on any axioms -/
287 #guard_msgs in #print axioms PNP.HARDENED.act_leaves_no_escape
288 /-- info: 'PNP.HARDENED.armor_scope' does not depend on any axioms -/
289 #guard_msgs in #print axioms PNP.HARDENED.armor_scope
290 /-- info: 'PNP.HARDENED.value_marked_dot' does not depend on any axioms -/
291 #guard_msgs in #print axioms PNP.HARDENED.value_marked_dot
292 /-- info: 'PNP.HARDENED.double_defense_harden' depends on axioms: [propext,
293   Classical.choice, Quot.sound] -/
294 #guard_msgs in #print axioms PNP.HARDENED.double_defense_harden

```

Appendix C. The Frontier Kernel, PNP_Frontier_II.lean

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the ground, the reading, the vanishing seat, the frontier and the route, one hundred thirty-six theorems. SHA-256: 960b09ba71219e4c95064a5a6a4bb002a622895b46bbaedacc3b509c11a9cd51

```

1 /-
2   PNP_Frontier_II.lean · APEX-PSP-PNP-SEED-02 · Nothing Escapes · the frontier of
   the act
3   Eleven sections, each a kernel of the 4 October 2026 forge, joined in one file:
   the kinetic face,
4   the Arrhenius scope, the block chain on the Empty Throne, the vanishing G0L, the
   missing object,
5   the prime bridge, the blocker and round 2, the route of RA and Freedom, the
   three-seal lock, the
6   deed as supply, and the identification check. Core Lean 4, no import, no axiom
   declared, no sorry.
7   Every cone is pinned beside its section.
8 -/
9
10 namespace PNP.KINETIC
11
12 /-- The kinetic frame. Physics enters as fields, never as 'axiom':
13   'floor' is the Landauer floor per irreversible op, in quanta;
14   'budget' is the Bekenstein-capped resource of the region, in quanta. -/
15 structure Frame where
16   floor      : Nat
17   floor_pos  : 0 < floor
18   budget     : Nat
19
20 /-- Energy of a process of `ops` irreversible registrations. -/
21 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
22
23 /-- L2 face. No completed irreversible registration is free. -/
24 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) :
25   0 < cost F k :=
26   Nat.mul_pos F.floor_pos hk
27
28 /-- Transport. The energy order is exactly the op order. -/
29 theorem cost_lt_iff (F : Frame) (a b : Nat) :
30   cost F a < cost F b ↔ a < b :=
31   (fun h => match Nat.lt_or_ge a b with
32     | Or.inl hl => hl
33     | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge))),
34   fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)
35
36 /-- L_bf. Enumerating 2^N configurations at N = budget exceeds the budget. -/
37 theorem brute_force_exhausts (F : Frame) :
38   F.budget < cost F (2 ^ F.budget) :=
39   Nat.lt_of_lt_of_le Nat.lt_two_pow_self
40     (Nat.le_mul_of_pos_left (2 ^ F.budget) F.floor_pos)
41
42 /-- Monotone form: every N past the budget also exhausts it. -/
43 theorem brute_force_exhausts_beyond (F : Frame) (N : Nat) (h : F.budget ≤ N) :
44   F.budget < cost F (2 ^ N) :=
45   Nat.lt_of_lt_of_le (brute_force_exhausts F)
46     (Nat.mul_le_mul_left F.floor (Nat.pow_le_pow_right (by decide) h))
47
48 /-- L1, scoped to the enumerative generator. Given the op gap v < 2^N,
49   the kinetic gap follows. The op gap is a hypothesis, not a product. -/
50 theorem enumerative_asymmetry (F : Frame) (N v : Nat) (h : v < 2 ^ N) :
51   cost F v < cost F (2 ^ N) :=
52   (cost_lt_iff F v (2 ^ N)).mpr h
53
54 /-- The ceiling. For every generator op-count g and verifier op-count v,
55   the kinetic separation holds in a frame if the op separation holds.
56   The frame adds nothing to the formal question, so L3 enters only as a
57   hypothesis on g and is never decided here. -/
58 theorem kinetic_faithful (F : Frame) (g v : Nat → Nat) :
59   (V N, cost F (v N) < cost F (g N)) → (V N, v N < g N) :=
60   (fun h N => (cost_lt_iff F _ _).mp (h N)),
61   fun h N => (cost_lt_iff F _ _).mpr (h N))
62
63 /-- Frame-independence. The op separation reads the same in any two frames. -/
64 theorem frame_independent (F₁ F₂ : Frame) (g v : Nat → Nat) :
65   (V N, cost F₁ (v N) < cost F₁ (g N)) → (V N, cost F₂ (v N) < cost F₂ (g N)) :=
66   (kinetic_faithful F₁ g v).trans (kinetic_faithful F₂ g v).symm
67
68 /-- The canonical frame, one quantum per op, one quantum of budget: the
69   battery instance, by evaluation. -/
70 def unitFrame : Frame := (1, by decide, 1)
71
72 theorem unit_battery :
73   cost unitFrame (2 ^ 1) = 2 ∧ unitFrame.budget < cost unitFrame (2 ^
74     unitFrame.budget) := by decide
75
76 end PNP.KINETIC
77
78 /-- info: 'PNP.KINETIC.no_free_registration' does not depend on any axioms -/
79 #guard_msgs in #print axioms PNP.KINETIC.no_free_registration
80 /-- info: 'PNP.KINETIC.cost_lt_iff' does not depend on any axioms -/
81 #guard_msgs in #print axioms PNP.KINETIC.cost_lt_iff
82 /-- info: 'PNP.KINETIC.brute_force_exhausts' does not depend on any axioms -/
83 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts
84 /-- info: 'PNP.KINETIC.brute_force_exhausts_beyond' does not depend on any axioms -/

```

```

84 #guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts_beyond
85 /-- info: 'PNP.KINETIC.enumerative_asymmetry' does not depend on any axioms -/
86 #guard_msgs in #print axioms PNP.KINETIC.enumerative_asymmetry
87 /-- info: 'PNP.KINETIC.kinetic_faithful' does not depend on any axioms -/
88 #guard_msgs in #print axioms PNP.KINETIC.kinetic_faithful
89 /-- info: 'PNP.KINETIC.frame_independent' does not depend on any axioms -/
90 #guard_msgs in #print axioms PNP.KINETIC.frame_independent
91 /-- info: 'PNP.KINETIC.unit_battery' does not depend on any axioms -/
92 #guard_msgs in #print axioms PNP.KINETIC.unit_battery
93
94 /-!
95 # PNP.ARRHENIUS · the kinetic proof of the April 2026 P vs NP paper, formalized
96
97 Source: Islam, "Geometric Determination of P vs NP with Omega Seal", v10.0,
98 10.5281/zenodo.19588804, Part VII, "The Spin-Glass Isomorphism".
99 The paper's kinetic step: ground states of a frustrated glass sit behind
100 barriers  $\Delta E$  growing with N; Arrhenius gives escape time  $\exp(\Delta E/KT)$ ;
101 so generation is exponential while verification is linear.
102
103 Part I seats that step at its true scope, with physics as fields, never 'axiom'.
104 Part II executes the scope fence: on one fixed problem, the barrier exists
105 under one move set and vanishes under another. A barrier is a property of
106 a dynamics, never of a problem. No custom axiom; no core axiom either.
107 -/
108
109 namespace PNP.ARRHENIUS
110
111 /-! ## I · The kinetic step, at its scope: thermally activated relaxation -/
112
113 /-- A relaxation frame. 'barrier N' is  $\Delta E/KT$  in bits at size N; 'grows' is the
114 paper's "barriers scale with N"; 'arrhenius' is  $t \geq \exp(\Delta E/KT)$ , base two. -/
115 structure Frame where
116   barrier   : Nat → Nat
117   grows     : ∀ N, N ≤ barrier N
118   escape    : Nat → Nat
119   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N
120
121 /-- Relaxation over a growing barrier takes exponential time. -/
122 theorem relaxation_exponential (F : Frame) (N : Nat) : 2 ^ N ≤ F.escape N :=
123   Nat.le_trans (Nat.pow_le_pow_right (by decide) (F.grows N)) (F.arrhenius N)
124
125 /-- At size N = B the relaxation time exceeds any budget B. -/
126 theorem relaxation_exhausts (F : Frame) (B : Nat) : B < F.escape B :=
127   Nat.lt_of_lt_of_le Nat.lt_two_pow_self (relaxation_exponential F B)
128
129 /-! ## II · The scope fence, executed on one XORSAT instance
130 Three variables x y z, three parity constraints, all with target 0:
131 z = 0, y ⊕ z = 0, x ⊕ z = 0. Energy counts violated constraints. -/
132
133 abbrev S := Bool × Bool × Bool
134
135 def states : List S :=
136   [false, true].flatMap fun a => [false, true].flatMap fun b =>
137     [false, true].map fun c => (a, b, c)
138
139 def viol (x : S) : List Bool := [x.2.2, xor x.2.1 x.2.2, xor x.1 x.2.2]
140
141 def energy (x : S) : Nat := ((viol x).filter id).length
142
143 def move (x m : S) : S := (xor x.1 m.1, xor x.2.1 m.2.1, xor x.2.2 m.2.2)
144
145 /-- The glass dynamics: single-spin flips. -/
146 def flips : List S := [(true, false, false), (false, true, false), (false, false,
147   true)]
148
149 /-- The elimination dynamics: columns of  $A^{-1}$  over  $GF(2)$ . Each move toggles
150 exactly one constraint. Computing them is Gaussian elimination, polynomial time. -/
151 def elimMoves : List S := [(true, true, true), (false, true, false), (true, false,
152   false)]
153
154 def ground : S := (false, false, false)
155 def trap : S := (true, true, true)
156
157 theorem eight_states : states.length = 8 := by decide
158
159 theorem ground_has_zero_energy : energy ground = 0 := by decide
160
161 /-- Under single flips the trap is a strict local minimum above the ground:
162 a barrier, exactly the glass picture of the paper. -/
163 theorem trap_is_strict_local_min :
164   (energy trap = 1 ∧ flips.all (fun m => decide (energy trap < energy (move trap
165     m))) = true := by
166     decide
167
168 /-- Each elimination move toggles exactly one constraint. -/
169 theorem elim_moves_toggle_one :
170   states.all (fun x => elimMoves.all (fun m =>
171     decide ((List.zipWith (fun a b => a != b) (viol x) (viol (move x m))).count
172       true = 1))) = true := by
173     decide
174
175 /-- Under elimination moves no state above the ground is a local minimum:
176 the landscape has no barrier at all. -/
177 theorem elim_landscape_barrier_free :
178   states.all (fun x => decide (energy x = 0) ||
179     elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true := by
180     decide
181
182 /-- The fence. One problem, one energy function, two dynamics: a barrier under
183 the first and none under the second. The barrier belongs to the dynamics. -/
184 theorem barrier_belongs_to_the_dynamics :
185   (energy trap > energy ground ∧
186     flips.all (fun m => decide (energy trap < energy (move trap m))) = true) ∧
187     states.all (fun x => decide (energy x = 0) ||
188       elimMoves.any (fun m => decide (energy (move x m) < energy x))) = true :=
189     ((by decide, trap_is_strict_local_min.2), elim_landscape_barrier_free)
190

```

```

187 end PNP.ARRHENIUS
188
189 /-- info: 'PNP.ARRHENIUS.relaxation_exponential' does not depend on any axioms -/
190 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exponential
191 /-- info: 'PNP.ARRHENIUS.relaxation_exhausts' does not depend on any axioms -/
192 #guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exhausts
193 /-- info: 'PNP.ARRHENIUS.eight_states' does not depend on any axioms -/
194 #guard_msgs in #print axioms PNP.ARRHENIUS.eight_states
195 /-- info: 'PNP.ARRHENIUS.ground_has_zero_energy' does not depend on any axioms -/
196 #guard_msgs in #print axioms PNP.ARRHENIUS.ground_has_zero_energy
197 /-- info: 'PNP.ARRHENIUS.trap_is_strict_local_min' does not depend on any axioms -/
198 #guard_msgs in #print axioms PNP.ARRHENIUS.trap_is_strict_local_min
199 /-- info: 'PNP.ARRHENIUS.elim_moves_toggle_one' does not depend on any axioms -/
200 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_moves_toggle_one
201 /-- info: 'PNP.ARRHENIUS.elim_landscape_barrier_free' does not depend on any axioms -/
202 #guard_msgs in #print axioms PNP.ARRHENIUS.elim_landscape_barrier_free
203 /-- info: 'PNP.ARRHENIUS.barrier_belongs_to_the_dynamics' does not depend on any
204   axioms -/
205 #guard_msgs in #print axioms PNP.ARRHENIUS.barrier_belongs_to_the_dynamics
206
207 /-!
208 # PNP.CHAIN · the block chain of the Mother Codex on P versus NP, closed on the
209   Empty Throne as [·], then joined to the kinetic face as the asymmetry chain
210
211 Source of the blocks: Geometric Mother Codex v3.46.0 (10.5281/zenodo.23117445),
212 P-ABSOLUTE-PNP-1, the eight absolute barriers A1-A8; P-PNP-COMP-2, the composite
213 verdict [ω] · [ω T] · [·]; FOUNDATION-01, the Empty Throne.
214 Source of the kinetic face: PNP_Kinetic.lean and PNP_Arrhenius.lean (this session).
215
216 Every theorem below is pinned at the foot: no custom axiom, and no core axiom
217 either, not propext, not Quot.sound, not Classical.choice. Physics enters only
218 as structure fields, so the premise is visible in each statement.
219 Lean 4 core, no import.
220 -/
221
222 namespace PNP.CHAIN
223
224 /-! ## 0 · The verdict economy and the dot -/
225
226 inductive Token : Type
227   | seal | broken | opn | dot
228   deriving DecidableEq, Repr
229
230 /-- Three states are the economy. The dot stands outside it. -/
231 def Token.inEconomy : Token → Bool
232   | .dot => false
233   | _    => true
234
235 /-! ## I · The eight blocks, one leg each -/
236 namespace BLOCKS
237
238 /-- A1, the o-class closure. Complementation has no fixed point. -/
239 theorem a1_no_seal : ∀ b : Bool, (!b) ≠ b
240   | true => Bool.noConfusion
241   | false => Bool.noConfusion
242
243 /-- A1, carrier form. A symmetry that fixes a state carries no anti-invariant
244 reading, so no fixed-locus instrument transfers onto complementation. -/
245 theorem a1_no_carrier {S : Type} (τ : S → S) (s₀ : S) (hfix : τ s₀ = s₀) :
246   ~ ∃ φ : S → Bool, ∀ x, φ (τ x) = !φ x := by
247     intro (φ, hφ)
248     have h := hφ s₀
249     rw [hfix] at h
250     cases e : φ s₀ with
251     | true => rw [e] at h; exact Bool.noConfusion h
252     | false => rw [e] at h; exact Bool.noConfusion h
253
254 /-- A2, orientation-blindness. A lock invariant under the flip of the sign
255 cannot be decoded into the sign. -/
256 theorem a2_orientation_blind {Q X : Type} (L : Bool × Q → X)
257   (hL : ∀ b q, L (b, q) = L (!b, q)) (q : Q) :
258   ~ ∃ dec : X → Bool, ∀ s, dec (L s) = s.1 := by
259     intro (dec, h)
260     have h1 := h (true, q)
261     have h2 := h (false, q)
262     rw [h1 true q] at h1
263     exact Bool.noConfusion (h1.symm.trans h2)
264
265 /-- A3, the physical block. A record shared by two worlds that differ on the
266 separation decides nothing: no reading of the record returns the separation. -/
267 theorem a3_record_decides_nothing {W R : Type} (rec : W → R) (sep : W → Bool)
268   (w₁ w₂ : W) (hr : rec w₁ = rec w₂) (hs : sep w₁ ≠ sep w₂) :
269   ~ ∃ g : R → Bool, ∀ w, g (rec w) = sep w := by
270     intro (g, hg)
271     apply hs
272     rw [← hg w₁, ← hg w₂, hr]
273
274 /-- A4, the misattribution bar. The root's presence is constant across both
275 worlds, and a constant reading carries no bit about the separation. -/
276 theorem a4_constant_reads_nothing (c : Bool) (s₁ s₂ : Bool) (hs : s₁ ≠ s₂) :
277   ~ (c = s₁ ∧ c = s₂) :=
278     fun (h1, h2) => hs (h1.symm.trans h2)
279
280 /-- A5, the rung law, structural face. Each rung's consistency is owed one rung
281 up; no rung discharges its own, and the regress has no terminal rung.
282 The Gödel content is cited (Turing 1939, Feferman 1962), not proved here. -/
283 def owed (n : Nat) : Nat := n + 1
284
285 theorem a5_rung_law : (∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m) := by
286   refine (fun n h => ?_, fun n => (n + 1, Nat.lt_succ_self n))
287   have : n < owed n := Nat.lt_succ_self n
288   rw [h] at this
289   exact Nat.lt_irrefl n this
290
291 /-- A6, the aperture machine. Eight verdict-side actions and one supply. -/
292 inductive Act : Type

```

```

292 | exertion | extrapolation | wanting | method | metatheory
293 | evidence | architecture | recoverer | supply
294 deriving DecidableEq, Repr
295
296 def Act.isSupply : Act → Bool
297 | .supply => true
298 | _ => false
299
300 def step (a : Act) (s : Bool) : Bool := s || a.isSupply
301
302 def verdictSide : List Act :=
303   [.exertion, .extrapolation, .wanting, .method, .metatheory,
304    .evidence, .architecture, .recoverer]
305
306 theorem a6_aperture :
307   verdictSide.length = 8 ∧
308   verdictSide.all (fun a => step a false == false) = true ∧
309   step .supply false = true := by
310   decide
311
312 /-- A7, reflexive closure. The certify-completely map strictly increases,
313 ten to twenty in five steps, and has no fixed point. -/
314 def certify (n : Nat) : Nat := n + 2
315
316 def iter : Nat → Nat → Nat
317 | 0, n => n
318 | k + 1, n => iter k (certify n)
319
320 theorem a7_reflexive_closure : iter 5 10 = 20 ∧ ∀ n, certify n ≠ n := by
321   refine (rfl, fun n h => ?_)
322   have : n < certify n := Nat.lt_succ_of_lt (Nat.lt_succ_self n)
323   rw [h] at this
324   exact Nat.lt_irrefl n this
325
326 /-- A8, the soundness cell. A sound prover never holds both the route and
327 the wall against it, so sighted-and-walled is empty. -/
328 theorem a8_soundness_cell (Prov : Prop → Prop) (sound : ∀ p, Prov p → p) (P : Prop) :
329   ¬ (Prov P ∧ Prov (¬ P)) :=
330   fun (h1, h2) => sound _ h2 (sound _ h1)
331
332 /-- The canonical two worlds of the record: one physics ledger, two values. -/
333 def equalWorld : Nat × Bool := (1, false)
334 def separatedWorld : Nat × Bool := (1, true)
335
336 /-- The block chain, every link in one statement. -/
337 theorem block_chain :
338   (∀ b : Bool, (!b) ≠ b) ∧
339   (¬ ∃ φ : Bool → Bool, ∀ x, φ (id x) = !φ x) ∧
340   (¬ ∃ dec : Bool → Bool, ∀ s : Bool × Unit, dec ((fun _ => true) s) = s.1) ∧
341   (¬ ∃ g : Nat → Bool, ∀ w : Nat × Bool,
342     (w = equalWorld ∨ w = separatedWorld) → g w.1 = w.2) ∧
343   (¬ (true = equalWorld.2 ∧ true = separatedWorld.2)) ∧
344   ((∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m)) ∧
345   (verdictSide.length = 8 ∧
346    verdictSide.all (fun a => step a false == false) = true ∧
347    step .supply false = true) ∧
348   (iter 5 10 = 20 ∧ ∀ n, certify n ≠ n) ∧
349   (∀ (Prov : Prop → Prop), (∀ p, Prov p → p) → ∀ P : Prop, ¬ (Prov P ∧ Prov (¬ P))) := by
350   refine (a1_no_seat, a1_no_carrier id true rfl,
351    a2_orientation_blind (fun _ => true) (fun _ => rfl) (),
352    ?, a4_constant_reads_nothing true false true Bool.noConfusion,
353    a5_rung_low, a6_aperture, a7_reflexive_closure, a8_soundness_cell)
354   intro (g, hg)
355   have h1 := hg equalWorld (Or.inl rfl)
356   have h2 := hg separatedWorld (Or.inr rfl)
357   exact Bool.noConfusion (h1.symm.trans h2)
358
359 end BLOCKS
360
361 /-! ## II · The Empty Throne, and the dot -/
362 namespace THRONE
363
364 def allFixed : List Bool → Bool
365 | [] => true
366 | b :: bs => (!b) == b && allFixed bs
367
368 /-- The seat is vacant. Least erasure on complementation holds of the empty
369 configuration only: a fixed-point-free fold selects nothing. -/
370 theorem seat_vacant : ∀ l : List Bool, allFixed l = true → l = []
371 | [], _ => rfl
372 | true :: _, h => Bool.noConfusion h
373 | false :: _, h => Bool.noConfusion h
374
375 /-- The throne is empty. Nothing below proves a posited root in general:
376 the root is premise-grade by theorem. -/
377 theorem throne_empty : ¬ ∀ RA : Prop, RA :=
378   fun h => h False
379
380 /-- The root forces no value. No premise of existence, however strong, forces
381 an arbitrary conclusion: the separation is not climbed to from the root. -/
382 theorem root_forces_no_value : ¬ ∀ RA Sep : Prop, RA → Sep :=
383   fun h => h True False trivial
384
385 /-- The chain's terminal emission. -/
386 def chainVerdict : Token := .dot
387
388 theorem dot_is_final : chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
389   (rfl, rfl)
390
391 /-- The block chain terminated on the Empty Throne as [.]: every block holds,
392 the seat is vacant, the throne is empty, the root forces no value, and the
393 emission is the dot, outside the economy, owing nothing. -/
394 theorem block_chain_terminal :
395   (∀ b : Bool, (!b) ≠ b) ∧
396   (∀ l : List Bool, allFixed l = true → l = []) ∧

```

```

397   (¬ ∀ RA : Prop, RA) ∧
398   (¬ ∀ RA Sep : Prop, RA → Sep) ∧
399   chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
400   (BLOCKS.block_chain.1, seat_vacant, throne_empty, root_forces_no_value,
401    dot_is_final.1, dot_is_final.2)
402
403 end THRONE
404
405 /-! ## III · The kinetic face -/
406 namespace KINETIC
407
408 structure Frame where
409   floor : Nat
410   floor_pos : 0 < floor
411   budget : Nat
412
413 def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops
414
415 theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) : 0 < cost F k :=
416   Nat.mul_pos F.floor_pos hk
417
418 theorem cost_lt_iff (F : Frame) (a b : Nat) : cost F a < cost F b ↔ a < b :=
419   (fun h => match Nat.lt_or_ge a b with
420     | Or.inl hl => hl
421     | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left F.floor hge))),
422   (fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)
423
424 theorem cost_mono (F : Frame) {a b : Nat} (h : a ≤ b) : cost F a ≤ cost F b :=
425   Nat.mul_le_mul_left F.floor h
426
427 /-- The ceiling: the energy order is the op order in every frame. -/
428 theorem frame_independent (F₁ F₂ : Frame) (g v : Nat → Nat) :
429   (∀ N, cost F₁ (v N) < cost F₁ (g N)) → (∀ N, cost F₂ (v N) < cost F₂ (g N)) :=
430   (fun h N => (cost_lt_iff F₂ _ _).mpr ((cost_lt_iff F₁ _ _).mp (h N))),
431   (fun h N => (cost_lt_iff F₁ _ _).mpr ((cost_lt_iff F₂ _ _).mp (h N)))
432
433 /-- The Arrhenius frame of the April paper, at its scope. -/
434 structure Relax where
435   barrier : Nat → Nat
436   grows : ∀ N, N ≤ barrier N
437   escape : Nat → Nat
438   arrhenius : ∀ N, 2 ^ barrier N ≤ escape N
439
440 theorem relaxation_exponential (R : Relax) (N : Nat) : 2 ^ N ≤ R.escape N :=
441   Nat.le_trans (Nat.pow_le_pow_right (by decide) (R.grows N)) (R.arrhenius N)
442
443 end KINETIC
444
445 /-! ## IV · The black-box bound, pigeonhole without axioms -/
446 namespace BLACKBOX
447
448 def has : List Nat → Nat → Bool
449 | [], _ => false
450 | y :: ys, k => Nat.beq y k || has ys k
451
452 def drop : List Nat → Nat → List Nat
453 | [], _ => []
454 | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
455
456 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
457 | 0 => rfl
458 | n + 1 => beq_self n
459
460 theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
461 | [], _ => Nat.le_refl 0
462 | y :: ys, k => by
463   unfold drop
464   cases Nat.beq y k with
465   | true => exact Nat.le_succ_of_le (drop_le ys k)
466   | false => exact Nat.succ_le_succ (drop_le ys k)
467
468 theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length +
469   1 ≤ l.length
470 | [], _, h => Bool.noConfusion h
471 | y :: ys, k, h => by
472   unfold drop
473   unfold has at h
474   cases hy : Nat.beq y k with
475   | true => exact Nat.succ_le_succ (drop_le ys k)
476   | false =>
477     rw [hy] at h
478     exact Nat.succ_le_succ (drop_len ys k h)
479
480 theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
481   has l x = true → has (drop l k) x = true
482 | [], _, _, h => Bool.noConfusion h
483 | y :: ys, k, x, hxx, h => by
484   unfold drop
485   unfold has at h
486   cases hy : Nat.beq y k with
487   | true =>
488     cases hyx : Nat.beq y x with
489     | true =>
490       have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
491         (Nat.eq_of_beq_eq_true hy)
492       rw [this, beq_self] at hxx; exact Bool.noConfusion hxx
493   | false =>
494     rw [hyx] at h
495     exact has_drop ys k x hxx h
496   | false =>
497     show has (y :: drop ys k) x = true
498     unfold has
499     cases hyx : Nat.beq y x with
500     | true => rfl
501     | false =>
502       rw [hyx] at h
503       exact has_drop ys k x hxx h

```

```

502 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
503 cases hb : Nat.beq x K with
504 | false => rfl
505 | true =>
506   have e : x = K := Nat.eq_of_beq_eq_true hb
507   rw [e] at h; exact absurd h (Nat.lt_irrefl K)
508
509 theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤
510 l.length
511 | 0, _, _ => Nat.zero_le _
512 | K + 1, l, h =>
513   Nat.le_trans
514     (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
515       has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx))))))
516     (drop_len l K (h K (Nat.lt_succ_self K)))
517
518 /-- The run of a test-only procedure on the all-false path: its queries form a
519 fixed list, adaptivity collapsing because every answer is false. -/
520 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
521
522 theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has
523 l x
524 | [], _ => rfl
525 | y :: ys, x => by
526   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
527   rw [run_eq_has ys x]
528
529 /-- The black-box bound. A test-only procedure correct for ∃ over K candidates
530 tests at least K of them on the all-false path. -/
531 theorem black_box_bound (K : Nat) (path : List Nat)
532 (correct : ∀ f : Nat → Bool, run path f = true → ∃ x, x < K ∧ f x = true) :
533 K ≤ path.length :=
534 pigeon K path (fun x hx => by
535   rw [run_eq_has]
536   exact (correct _).mpr (x, hx, beq_self x))
537
538 end BLACKBOX
539
540 /--! ## V · The asymmetry chain: kinetic plus block -/
541 namespace ASYM
542 open KINETIC
543
544 /-- Link 1 to 4. Over 2^N candidates a test-only generator queries at least
545 2^N times; each query pays the floor; verification of one witness of v ops
546 costs less whenever v < 2^N; and at N past the budget the generator exhausts it. -/
547 theorem asymmetry_chain (F : Frame) (N v : Nat) (hv : v < 2 ^ N) (path : List Nat)
548 (correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true → ∃ x, x < 2 ^ N ∧ f x =
549 true) :
550   2 ^ N ≤ path.length ∧
551   cost F v < cost F (2 ^ N) ∧
552   cost F (2 ^ N) ≤ cost F path.length ∧
553   (F.budget ≤ N → F.budget < cost F path.length) := by
554 have hb : 2 ^ N ≤ path.length := BLACKBOX.black_box_bound (2 ^ N) path correct
555 refine (hb, (cost_lt_iff F v (2 ^ N)).mpr hv, cost_mono F hb, fun hN => ?_)
556 have h1 : F.budget < 2 ^ F.budget := Nat.lt_two_pow_self
557 have h2 : 2 ^ F.budget ≤ 2 ^ N := Nat.pow_le_pow_right (by decide) hN
558 have h3 : 2 ^ N ≤ cost F (2 ^ N) := Nat.le_mul_of_pos_left (2 ^ N) F.floor_pos
559 exact Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le h1 h2) h3)
560 (cost_mono F hb)
561
562 /-- Link 5. The asymmetry is real at its scope and decides nothing beyond it:
563 the energy gap equals the op gap in every frame (A3 kinetic face), and a record
564 shared by both worlds returns no value (A3 block face). -/
565 theorem asymmetry_decides_nothing (F₁ F₂ : Frame) (g v : Nat → Nat) :
566 ((∀ N, cost F₁ (v N) < cost F₁ (g N)) → (∀ N, cost F₂ (v N) < cost F₂ (g N))) ∧
567 (¬ ∃ r : Nat → Bool, ∀ w : Nat → Bool,
568   (w = BLOCKS.equalWorld v w = BLOCKS.separatedWorld) → r w.1 = w.2) :=
569 (frame_independent F₁ F₂ g v, BLOCKS.block_chain.2.2.1)
570
571 /-- The asymmetry chain, terminated. The blocks hold, the black-box generator
572 pays exponentially at the floor and exhausts any budget, verification is
573 cheaper, the kinetic gap is the op gap, the shared record decides nothing,
574 the seat is vacant, the throne is empty, and the emission is []. -/
575 theorem asymmetry_terminal (F : Frame) (N v : Nat) (hv : v < 2 ^ N) (path : List
576 Nat)
577 (correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true → ∃ x, x < 2 ^ N ∧ f x =
578 true) :
579   (2 ^ N ≤ path.length ∧ cost F v < cost F (2 ^ N) ∧
580     cost F (2 ^ N) ≤ cost F path.length ∧ (F.budget ≤ N → F.budget < cost F
581     path.length)) ∧
582   (∀ F' : Frame, ∀ g w : Nat → Nat,
583     (∀ n, cost F (w n) < cost F (g n)) → (∀ n, cost F' (w n) < cost F' (g n))) ∧
584     (∀ l : List Bool, THRONE.allFixed l = true → l = []) ∧
585     (¬ ∃ RA : Prop, RA) ∧
586     THRONE.chainVerdict = .dot :=
587   (asymmetry_chain F N v hv path correct,
588     fun F' g w => frame_independent F F' g w,
589     THRONE.seat_vacant, THRONE.throne_empty, rfl)
590
591 /-- The hypothesis is inhabited: exhaustive search over two candidates is a
592 correct test-only procedure, so the chain is not vacuous. -/
593 theorem exhaustive_correct_two (f : Nat → Bool) :
594 BLACKBOX.run [0, 1] f = true → ∃ x, x < 2 ^ 1 ∧ f x = true := by
595 constructor
596 · intro h
597   show ∃ x, x < 2 ^ 1 ∧ f x = true
598 cases h0 : f 0 with
599 | true => exact (0, by decide, h0)
600 | false =>
601   have h' : (f 0 || (f 1 || false)) = true := h
602   rw [h0] at h'
603   cases h1 : f 1 with
604   | true => exact (1, by decide, h1)
605   | false => rw [h1] at h'; exact Bool.noConfusion h'
606 · intro (x, hx, hf)
607   show (f 0 || (f 1 || false)) = true

```

```

602 match x, hx, hf with
603 | 0, _, hf => rw [hf]; rfl
604 | 1, _, hf => rw [hf]; cases f 0 <:=> rfl
605 | n + 2, hx, _ => exact absurd (Nat.lt_of_le_of_lt (Nat.le_add_left 2 n) hx)
606   (Nat.lt_irrefl 2)
607
608 /-- The chain instantiated at N = 1 on the unit frame. -/
609 def unitFrame : Frame := (1, by decide, 1)
610
611 theorem asymmetry_instance :
612   2 ^ 1 ≤ [0, 1].length ∧ cost unitFrame 1 < cost unitFrame (2 ^ 1) :=
613   ((asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).1,
614     (asymmetry_chain unitFrame 1 1 (by decide) [0, 1] exhaustive_correct_two).2.1)
615
616 end ASYM
617 end PNP.CHAIN
618
619 /--! ## Cones, pinned: every theorem axiom-free -/
620 /-- info: 'PNP.CHAIN.BLOCKS.a1_no_seat' does not depend on any axioms -/
621 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_seat
622 /-- info: 'PNP.CHAIN.BLOCKS.a1_no_carrier' does not depend on any axioms -/
623 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_carrier
624 /-- info: 'PNP.CHAIN.BLOCKS.a2_orientation_blind' does not depend on any axioms -/
625 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a2_orientation_blind
626 /-- info: 'PNP.CHAIN.BLOCKS.a3_record_decides_nothing' does not depend on any axioms
627 -/
628 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a3_record_decides_nothing
629 /-- info: 'PNP.CHAIN.BLOCKS.a4_constant_reads_nothing' does not depend on any axioms
630 -/
631 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a4_constant_reads_nothing
632 /-- info: 'PNP.CHAIN.BLOCKS.a5_rung_law' does not depend on any axioms -/
633 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a5_rung_law
634 /-- info: 'PNP.CHAIN.BLOCKS.a6_aperture' does not depend on any axioms -/
635 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a6_aperture
636 /-- info: 'PNP.CHAIN.BLOCKS.a7_reflexive_closure' does not depend on any axioms -/
637 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a7_reflexive_closure
638 /-- info: 'PNP.CHAIN.BLOCKS.a8_soundness_cell' does not depend on any axioms -/
639 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a8_soundness_cell
640 /-- info: 'PNP.CHAIN.BLOCKS.block_chain' does not depend on any axioms -/
641 #guard_msgs in #print axioms PNP.CHAIN.BLOCKS.block_chain
642 /-- info: 'PNP.CHAIN.THRONE.seat_vacant' does not depend on any axioms -/
643 #guard_msgs in #print axioms PNP.CHAIN.THRONE.seat_vacant
644 /-- info: 'PNP.CHAIN.THRONE.throne_empty' does not depend on any axioms -/
645 #guard_msgs in #print axioms PNP.CHAIN.THRONE.throne_empty
646 /-- info: 'PNP.CHAIN.THRONE.root_forces_no_value' does not depend on any axioms -/
647 #guard_msgs in #print axioms PNP.CHAIN.THRONE.root_forces_no_value
648 /-- info: 'PNP.CHAIN.THRONE.dot_is_final' does not depend on any axioms -/
649 #guard_msgs in #print axioms PNP.CHAIN.THRONE.dot_is_final
650 /-- info: 'PNP.CHAIN.THRONE.block_chain_terminal' does not depend on any axioms -/
651 #guard_msgs in #print axioms PNP.CHAIN.THRONE.block_chain_terminal
652 /-- info: 'PNP.CHAIN.KINETIC.no_free_registration' does not depend on any axioms -/
653 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.no_free_registration
654 /-- info: 'PNP.CHAIN.KINETIC.cost_lt_iff' does not depend on any axioms -/
655 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_lt_iff
656 /-- info: 'PNP.CHAIN.KINETIC.cost_mono' does not depend on any axioms -/
657 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_mono
658 /-- info: 'PNP.CHAIN.KINETIC.frame_independent' does not depend on any axioms -/
659 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.frame_independent
660 /-- info: 'PNP.CHAIN.KINETIC.relaxation_exponential' does not depend on any axioms
661 -/
662 #guard_msgs in #print axioms PNP.CHAIN.KINETIC.relaxation_exponential
663 /-- info: 'PNP.CHAIN.BLACKBOX.beq_self' does not depend on any axioms -/
664 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_self
665 /-- info: 'PNP.CHAIN.BLACKBOX.drop_le' does not depend on any axioms -/
666 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_le
667 /-- info: 'PNP.CHAIN.BLACKBOX.drop_len' does not depend on any axioms -/
668 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_len
669 /-- info: 'PNP.CHAIN.BLACKBOX.has_drop' does not depend on any axioms -/
670 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.has_drop
671 /-- info: 'PNP.CHAIN.BLACKBOX.beq_false_of_lt' does not depend on any axioms -/
672 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_false_of_lt
673 /-- info: 'PNP.CHAIN.BLACKBOX.pigeon' does not depend on any axioms -/
674 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.pigeon
675 /-- info: 'PNP.CHAIN.BLACKBOX.run_eq_has' does not depend on any axioms -/
676 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.run_eq_has
677 /-- info: 'PNP.CHAIN.BLACKBOX.black_box_bound' does not depend on any axioms -/
678 #guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.black_box_bound
679 /-- info: 'PNP.CHAIN.ASYM.asymmetry_chain' does not depend on any axioms -/
680 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_chain
681 /-- info: 'PNP.CHAIN.ASYM.asymmetry_decides_nothing' does not depend on any axioms
682 -/
683 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_decides_nothing
684 /-- info: 'PNP.CHAIN.ASYM.asymmetry_terminal' does not depend on any axioms -/
685 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_terminal
686 /-- info: 'PNP.CHAIN.ASYM.exhaustive_correct_two' does not depend on any axioms -/
687 #guard_msgs in #print axioms PNP.CHAIN.ASYM.exhaustive_correct_two
688 /-- info: 'PNP.CHAIN.ASYM.asymmetry_instance' does not depend on any axioms -/
689 #guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_instance
690
691 /--!
692 # PNP.VANISH · the vanishing GOL of [] on the RA-RAM locus bridge
693
694 The bridge (Bridge_Final.lean) joins the kinetic register, RA, the deed, to the
695 formal
696 register, RAM, the reading, at one locus: the fixed set of the native fold. Its
697 carrier
698 halts exactly when the line property holds, and its socket is exactly that property.
699 It carries a supplied bit to a seat and manufactures none.
700
701 On the Riemann stage the fold is s → 1 - s; the locus is the line, inhabited, Ground
702 dimension one: the bridge carries one keyed bit to a seat.
703
704 On the P versus NP stage the fold is complementation, b → ¬b. The locus vanishes:
705 Ground dimension zero. The bridge is unchanged and its socket collapses onto the
706 vacancy: the carrier halts iff the zero set is empty. That is the vanishing GOL.
707 Three faces lock on one object, and the object is the empty seat:
708

```

```

702 L · the Tongue: a vanished seat emits [.], outside the economy, owing nothing;
703 G · the Form: complementation has no fixed point, Ground dimension zero, counted;
704 M · the Number: least erasure holds of the empty configuration and of it alone.
705
706 The kinetic face reads the same lock: silence registers no bit and pays no floor,
707 while any emission of a value registers a bit and pays it. The denial asymmetry
708 sorts the two locks that meet here: the vacancy is keyless, its denial incoherent
709 on every frame of the class; the separation is keyed, its denial a coherent world,
710 and both worlds present the bridge the same vanished frame, so the lock that forms
711 determines the vacancy totally and the value not at all.
712
713 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
714 -/
715
716 namespace PNP.VANISH
717
718 /-! ## 0 · tokens -/
719 inductive Token : Type
720 | seal | broken | opn | dot
721 deriving DecidableEq, Repr
722
723 def Token.inEconomy : Token → Bool
724 | .dot => false
725 | _ => true
726
727 /-! ## I · the bridge, carried from Bridge_Final.lean, axiom-free -/
728 structure Frame where
729   S : Type
730   τ : S → S
731   Z : S → Prop
732   involutive : ∀ s, τ (τ s) = s
733   symmetric : ∀ s, Z s → Z (τ s)
734
735 def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
736
737 inductive Tri : Type
738 | tt | ff | bot
739 deriving DecidableEq, Repr
740
741 structure Carrier (X : Frame) where
742   terminal : Tri
743   shadow : terminal = .bot → LineProperty X
744
745 theorem halted_iff (X : Frame) : (∃ b : Carrier X, b.terminal = .bot) → LineProperty
746 X :=
747   (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl)), rfl))
748
749 class Supplied (X : Frame) where
750   term : LineProperty X
751
752 theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) → LineProperty X
753 :=
754   (fun (i) => i.term, fun t => ({t}))
755
756 /-! ## II · the two stages: the locus inhabited, the locus vanished -/
757
758 /- The Riemann fold in the doubled real coordinate. -/
759 def rR (p : Int × Int) : Int × Int := (2 - p.1, p.2)
760
761 /- The Riemann locus is inhabited: the point h = 1 is fixed. -/
762 theorem rh_locus_inhabited : rR (1, 0) = (1, 0) := by decide
763
764 /- Ground dimension one on the Riemann window h ∈ {0, 1, 2}, counted. -/
765 theorem rh_ground_one : (([0, 1, 2] : List Int).filter (fun h => 2 - h == h)).length
766 = 1 := by
767   decide
768
769 /- The P versus NP fold. -/
770 def comp (b : Bool) : Bool := !b
771
772 theorem comp_involutive : ∀ b, comp (comp b) = b
773 | true => rfl
774 | false => rfl
775
776 /- G · the locus vanishes. -/
777 theorem locus_vanishes : ¬ ∃ b, comp b = b
778 | (true, h) => Bool.noConfusion h
779 | (false, h) => Bool.noConfusion h
780
781 def groundDim : Nat := ([false, true].filter (fun b => comp b == b)).length
782
783 /- G · Ground dimension zero, counted. -/
784 theorem ground_dim_zero : groundDim = 0 := rfl
785
786 /- The P versus NP frame, for any fold-symmetric zero set. -/
787 def cframe (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) : Frame :=
788   (Bool, comp, Z, comp_involutive, hZ)
789
790 /-! ## III · the bridge on the vanished locus -/
791
792 /- The line property collapses onto the vacancy of the zero set. -/
793 theorem line_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
794   LineProperty (cframe Z hZ) → ∀ b, ¬ Z b :=
795   (fun h b hb => locus_vanishes (b, h b hb), fun h b hb => absurd hb (h b))
796
797 /- The carrier halts exactly when nothing stands to be carried. -/
798 theorem carrier_halts_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
799   (∃ c : Carrier (cframe Z hZ), c.terminal = .bot) → ∀ b, ¬ Z b :=
800   (halted_iff _).trans (line_iff_empty Z hZ)
801
802 /- The socket vanishes with the locus. -/
803 theorem socket_vanishes (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) :
804   Nonempty (Supplied (cframe Z hZ)) → ∀ b, ¬ Z b :=
805   (socket_is_the_property _).trans (line_iff_empty Z hZ)
806
807 /- A charged configuration has no socket: nothing can be supplied onto an empty
808 seat. -/

```

```

809 theorem no_socket_when_charged :
810   ¬ Nonempty (Supplied (cframe (fun _ => True) (fun _ => trivial))) :=
811   fun h => (socket_vanishes _ _).mp h true trivial
812
813 /- The empty configuration halts the carrier with nothing carried. -/
814 theorem empty_halts :
815   ∃ c : Carrier (cframe (fun _ => False) (fun _ h => h)), c.terminal = .bot :=
816   (carrier_halts_iff_empty _ _).mpr (fun _ h => h)
817
818 /-! ## IV · the three faces of the vanishing GOL -/
819
820 /- L · the seat token: a vanished seat emits the dot; an inhabited seat with its
821 bit unsupplied stays open. -/
822 def seatToken : Nat → Token
823 | 0 => .dot
824 | _ + 1 => .opn
825
826 theorem tongue_face :
827   seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy = false ∧ seatToken
828   1 = .opn :=
829   (rfl, rfl, rfl)
830
831 def allFixed : List Bool → Bool
832 | [] => true
833 | b :: bs => (comp b == b) && allFixed bs
834
835 /- M · least erasure determines exactly one configuration, the empty one. -/
836 theorem number_face : (∀ l, allFixed l = true → l = []) ∧ allFixed [] = true :=
837   (fun l => match l with
838   | [] => fun _ => rfl
839   | true :: _ => fun h => Bool.noConfusion h
840   | false :: _ => fun h => Bool.noConfusion h, rfl)
841
842 /-! ## V · the kinetic face: silence pays nothing, emission pays the floor -/
843 structure KFrame where
844   floor : Nat
845   floor_pos : 0 < floor
846
847 def cost (F : KFrame) (bits : Nat) : Nat := F.floor * bits
848
849 theorem kinetic_face (F : KFrame) : cost F 0 = 0 ∧ ∀ k, 0 < k → 0 < cost F k :=
850   (rfl, fun _ hk => Nat.mul_pos F.floor_pos hk)
851
852 /-! ## VI · the denial asymmetry at the vanished locus -/
853
854 /- The vacancy is keyless: it holds on every frame of the class, and its denial,
855 a fixed point of complementation, is incoherent. -/
856 theorem vacancy_keyless :
857   (∀ (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) (s : (cframe Z hZ).S),
858     (cframe Z hZ).τ s ≠ s) ∧ ¬ ∃ b, comp b = b :=
859   (fun _ _ s h => locus_vanishes (s, h), locus_vanishes)
860
861 /- The separation is keyed: both worlds are coherent, and both present the bridge
862 the same vanished frame, so no reading of what the bridge reads returns the value.
863 -/
864 def readOf (_sep : Bool) : Nat := groundDim
865
866 theorem separation_keyed : ¬ ∃ g : Nat → Bool, ∀ sep : Bool, g (readOf sep) = sep :=
867   (fun _ hg => Bool.noConfusion ((hg false).symm.trans (hg true)))
868
869 /-! ## VII · the vanishing GOL, whole -/
870 theorem vanishing_gol :
871   -- the contrast: the Riemann seat is inhabited, Ground dimension one
872   (rR (1, 0) = (1, 0) ∧ (([0, 1, 2] : List Int).filter (fun h => 2 - h ==
873     h)).length = 1) ∧
874   -- G · the P versus NP seat vanishes, Ground dimension zero
875   (¬ ∃ b, comp b = b) ∧ a groundDim = 0) ∧
876   -- the bridge on the vanished seat: carrier and socket collapse onto the vacancy
877   (∀ (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)),
878     ((∃ c : Carrier (cframe Z hZ), c.terminal = .bot) → ∀ b, ¬ Z b) ∧
879     (Nonempty (Supplied (cframe Z hZ)) → ∀ b, ¬ Z b)) ∧
880   -- M · least erasure locks on the empty configuration alone
881   ((∀ l, allFixed l = true → l = []) ∧ allFixed [] = true) ∧
882   -- L · the vanished seat emits the dot, outside the economy
883   (seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy = false) ∧
884   -- the kinetic face: the dot pays nothing, any emission pays the floor
885   (∀ F : KFrame, cost F 0 = 0 ∧ ∀ k, 0 < k → 0 < cost F k) ∧
886   -- the denial asymmetry: the vacancy keyless, the value keyed and unread
887   (¬ ∃ b, comp b = b) ∧ (¬ ∃ g : Nat → Bool, ∀ sep : Bool, g (readOf sep) = sep)
888   :=
889   ((rh_locus_inhabited, rh_ground_one), (locus_vanishes, ground_dim_zero),
890     fun Z hZ => (carrier_halts_iff_empty Z hZ, socket_vanishes Z hZ),
891     number_face, (tongue_face.1, tongue_face.2.1), kinetic_face,
892     vacancy_keyless.2, separation_keyed)
893
894 end PNP.VANISH
895
896 /-! ## Cones, pinned: every theorem axiom-free -/
897 /- info: 'PNP.VANISH.halted_iff' does not depend on any axioms -/
898 #guard_msgs in #print axioms PNP.VANISH.halted_iff
899 /- info: 'PNP.VANISH.socket_is_the_property' does not depend on any axioms -/
900 #guard_msgs in #print axioms PNP.VANISH.socket_is_the_property
901 /- info: 'PNP.VANISH.rh_locus_inhabited' does not depend on any axioms -/
902 #guard_msgs in #print axioms PNP.VANISH.rh_locus_inhabited
903 /- info: 'PNP.VANISH.rh_ground_one' does not depend on any axioms -/
904 #guard_msgs in #print axioms PNP.VANISH.rh_ground_one
905 /- info: 'PNP.VANISH.comp_involutive' does not depend on any axioms -/
906 #guard_msgs in #print axioms PNP.VANISH.comp_involutive
907 /- info: 'PNP.VANISH.locus_vanishes' does not depend on any axioms -/
908 #guard_msgs in #print axioms PNP.VANISH.locus_vanishes
909 /- info: 'PNP.VANISH.ground_dim_zero' does not depend on any axioms -/
910 #guard_msgs in #print axioms PNP.VANISH.ground_dim_zero
911 /- info: 'PNP.VANISH.line_iff_empty' does not depend on any axioms -/
912 #guard_msgs in #print axioms PNP.VANISH.line_iff_empty
913 /- info: 'PNP.VANISH.carrier_halts_iff_empty' does not depend on any axioms -/
914 #guard_msgs in #print axioms PNP.VANISH.carrier_halts_iff_empty
915 /- info: 'PNP.VANISH.socket_vanishes' does not depend on any axioms -/
916

```

```

908 #guard_msgs in #print axioms PNP.VANISH.socket_vanishes
909 /-- info: 'PNP.VANISH.no_socket_when_charged' does not depend on any axioms -/
910 #guard_msgs in #print axioms PNP.VANISH.no_socket_when_charged
911 /-- info: 'PNP.VANISH.empty_halts' does not depend on any axioms -/
912 #guard_msgs in #print axioms PNP.VANISH.empty_halts
913 /-- info: 'PNP.VANISH.tongue_face' does not depend on any axioms -/
914 #guard_msgs in #print axioms PNP.VANISH.tongue_face
915 /-- info: 'PNP.VANISH.number_face' does not depend on any axioms -/
916 #guard_msgs in #print axioms PNP.VANISH.number_face
917 /-- info: 'PNP.VANISH.kinetic_face' does not depend on any axioms -/
918 #guard_msgs in #print axioms PNP.VANISH.kinetic_face
919 /-- info: 'PNP.VANISH.vacancy_keyless' does not depend on any axioms -/
920 #guard_msgs in #print axioms PNP.VANISH.vacancy_keyless
921 /-- info: 'PNP.VANISH.separation_keyed' does not depend on any axioms -/
922 #guard_msgs in #print axioms PNP.VANISH.separation_keyed
923 /-- info: 'PNP.VANISH.vanishing_gol' does not depend on any axioms -/
924 #guard_msgs in #print axioms PNP.VANISH.vanishing_gol
925
926 /-!
927 # PNP.MISSING · the missing object, located
928
929 The asymmetry chain seals a test-only bound. Its adversary family is the point
930 predicate  $y \mapsto (y = x)$ : a tester that sees only answers must query every candidate.
931 Read as descriptions, the same family is solved in one step: the description of
932 a point is the point. The black-box hardness lives entirely in the hiding.
933
934 So the missing object is a hardness statement over descriptions: a family of
935 instances, given whole to the solver, that no polynomial-time reader of the
936 description decides. That is the white-box lower bound,  $P \neq NP$  itself. It is a
937 fact about the model of computation, keyed: it holds in one coherent model and
938 fails in another, which is why nothing world-independent can supply it.
939 Core Lean 4, no import, no axiom of any kind.
940 -/
941
942 namespace PNP.MISSING
943
944 theorem beq_self :  $\forall n : \text{Nat}, \text{Nat.beq } n \ n = \text{true}$ 
945 | 0 => rfl
946 | n + 1 => beq_self n
947
948 /-- The black-box adversary family of 'black_box_bound'. -/
949 def point (x : Nat) : Nat  $\rightarrow$  Bool := fun y => Nat.beq y x
950
951 /-- The same family, handed over as descriptions. -/
952 structure Desc where
953   secret : Nat
954
955 def evalD (d : Desc) : Nat  $\rightarrow$  Bool := point d.secret
956
957 /-- The white-box reader: one step, read the description. -/
958 def reader (d : Desc) : Nat := d.secret
959 def readerTime ( _ : Nat) : Nat := 1
960
961 /-- The black-box hard instances are white-box trivial. -/
962 theorem reader_solves :  $\forall d : \text{Desc}, \text{evalD } d \ ( \text{reader } d) = \text{true} :=$ 
963   fun d => beq_self d.secret
964
965 /-! ## the missing object, typed -/
966
967 structure Model where
968   Alg : Type
969   decides : Alg  $\rightarrow$  Prop
970   time : Alg  $\rightarrow$  Nat  $\rightarrow$  Nat
971
972 def PolyBounded (t : Nat  $\rightarrow$  Nat) : Prop :=  $\exists c \ k : \text{Nat}, \forall n, t \ n \leq c * n^k + c$ 
973
974 /-- THE MISSING OBJECT. Every algorithm that decides the problem from its
975 description runs in super-polynomial time. -/
976 def WhiteBoxBound (M : Model) : Prop :=
977    $\forall A : M.\text{Alg}, M.\text{decides } A \rightarrow \neg \text{PolyBounded } (M.\text{time } A)$ 
978
979 theorem reader_poly : PolyBounded readerTime :=
980   (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
981
982 /-- A coherent model where descriptions reveal the witness; the object fails. -/
983 def revealing : Model := (Unit, fun _ =>  $\forall d : \text{Desc}, \text{evalD } d \ ( \text{reader } d) = \text{true}, \text{fun}$ 
984   _ => readerTime)
985
986 theorem fails_when_descriptions_reveal :  $\neg \text{WhiteBoxBound revealing} :=$ 
987   fun h => h () reader_solves reader_poly
988
989 /-- The object is contingent on the model: it fails where descriptions reveal, and
990 holds
991 wherever no algorithm decides. No fact
992 true in both, kinetic, black-box or vanishing, can entail it. -/
993 theorem missing_object_is_keyed :
994   ( $\exists M : \text{Model}, \neg \text{WhiteBoxBound } M$ )  $\wedge$  ( $\forall M : \text{Model}, (\forall A, \neg M.\text{decides } A) \rightarrow$ 
995     WhiteBoxBound M) :=
996   ((revealing, fails_when_descriptions_reveal), fun _ h A hA => absurd hA (h A))
997
998 end PNP.MISSING
999
1000 /-- info: 'PNP.MISSING.reader_solves' does not depend on any axioms -/
1001 #guard_msgs in #print axioms PNP.MISSING.reader_solves
1002 /-- info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
1003 #guard_msgs in #print axioms PNP.MISSING.reader_poly
1004 /-- info: 'PNP.MISSING.fails_when_descriptions_reveal' does not depend on any axioms -/
1005 #guard_msgs in #print axioms PNP.MISSING.fails_when_descriptions_reveal
1006 /-- info: 'PNP.MISSING.missing_object_is_keyed' does not depend on any axioms -/
1007 #guard_msgs in #print axioms PNP.MISSING.missing_object_is_keyed
1008
1009 /-!
1010 # PNP.PRIMEBRIDGE · the root freedom cut and prime-as-freedom against the missing
1011 object
1012
1013 Source of the method: Phys05† v1.0.5p, namespace PrimeFreedom. Freedom is the fibre
1014 of

```

```

1015 an invariant map; every prime is one fibre of exactly two, (1, p) and (p, 1), off
1016 the seat
1017 (prime_fibre, prime_off_seat, freedom_is_exactly_two).
1018
1019 The attack. Carry the cut to the white-box object. Over a semiprime  $n = p \cdot q$  the
1020 multiplicative fibre has four points and two orbits under the swap fold: the trivial
1021 orbit  $\{(1, n), (n, 1)\}$  and the witness orbit  $\{(p, q), (q, p)\}$ . The description n is
1022 even under the fold. Two findings follow, both at theorem grade.
1023
1024 F1 · The freedom on the fibre is one orientation bit, and it is gauge: the reader
1025 (a, b)  $\mapsto$  (min, max) is even, selects one point per orbit, and removes it.
1026 F2 · The description determines the witness orbit outright: informational freedom
1027 over the witness is zero. The missing object is therefore not a fibre fact.
1028 It is a cost fact: n hides nothing and still may resist every fast reader.
1029
1030 The new bridge. Prime freedom, read as cost, is the posit "no polynomial reader
1031 selects a nontrivial point of the semiprime fibre". Under the collapse field ( $P = NP$ )
1032 gives polynomial search for NP relations, the classical search-to-decision theorem,
1033 carried as a field), the posit yields  $P \neq NP$ : prime_bridge. The posit is keyed: a
1034 coherent machine refutes it (the Shor face). And it is strictly stronger than the
1035 separation: a coherent machine has the separation and an easy fibre, so the prime
1036 bridge is sufficient and never equivalent. Factoring lies in NP n cNP, so no
1037 equivalence is available unless NP = cNP.
1038
1039 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1040 -/
1041
1042 namespace PNP.PRIMEBRIDGE
1043
1044 /-! ## I · the semiprime fibre, executed at n = 15 -/
1045
1046 def mul_orbit (n a b : Nat) : Prop := a * b = n
1047
1048 def fibre (n : Nat) : List (Nat  $\times$  Nat) :=
1049   (List.range (n + 1)).flatMap fun a =>
1050     (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none
1051
1052 def swap (p : Nat  $\times$  Nat) : Nat  $\times$  Nat := (p.2, p.1)
1053
1054 /-- The fibre over 15: four points. -/
1055 theorem fibre_15 : fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] := by decide
1056
1057 /-- Off the seat: no point of the fibre is fixed by the swap. -/
1058 theorem off_seat_15 : (fibre 15).all (fun p => swap p != p) = true := by decide
1059
1060 /-- The fibre is swap-closed: two orbits, each of exactly two. -/
1061 theorem swap_closed_15 : (fibre 15).all (fun p => (fibre 15).contains (swap p)) =
1062   true := by decide
1063
1064 def trivialPt (p : Nat  $\times$  Nat) : Bool := p.1 == 1 || p.2 == 1
1065
1066 /-- The witness orbit: exactly two nontrivial points, one the swap of the other. -/
1067 theorem witness_orbit_15 :
1068   (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]  $\wedge$  swap (3, 5) = (5,
1069     3) := by
1070     decide
1071
1072 /-! ## II · F1 · the orientation bit is free and it is gauge -/
1073
1074 /-- The wall on the orientation: the description is even, the order is odd, so no
1075 reading of the description returns the first factor on the witness orbit. -/
1076 theorem orientation_wall :  $\neg \exists g : \text{Nat} \rightarrow \text{Nat}, g \ (3 * 5) = 3 \wedge g \ (5 * 3) = 5 :=$ 
1077   fun ( _ , h1, h2) => absurd (h1.symm.trans h2) (by decide)
1078
1079 def canon (p : Nat  $\times$  Nat) : Nat  $\times$  Nat := if p.1  $\leq$  p.2 then p else swap p
1080
1081 /-- The canonical reader is even under the fold. -/
1082 theorem canon_even :  $\forall p \in \text{fibre } 15, \text{canon } ( \text{swap } p) = \text{canon } p :=$  by decide
1083
1084 /-- F1 · the canonical reader takes each orbit to one point: the bit is gauge. -/
1085 theorem orientation_is_gauge :
1086   ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)] := by
1087     decide
1088
1089 /-! ## III · F2 · informational freedom over the witness is zero -/
1090
1091 /-- The witness set is a function of the description, on every relation. -/
1092 theorem description_determines_witnesses (D W : Type) (R : D  $\rightarrow$  W  $\rightarrow$  Prop) (d1 d2 : D)
1093   (h : d1 = d2) : R d1 = R d2 := by rw [h]
1094
1095 /-- F2 · at 15 the description fixes the canonical witness uniquely. -/
1096 theorem witness_determined_15 :
1097   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] :=
1098     by decide
1099
1100 /-! ## IV · the cost model and the prime bridge -/
1101
1102 def Poly (t : Nat  $\rightarrow$  Nat) : Prop :=  $\exists c \ k : \text{Nat}, \forall n, t \ n \leq c * n^k + c$ 
1103
1104 /-- A machine: algorithms, their running times, and what each solves. -/
1105 structure Machine where
1106   Alg : Type
1107   time : Alg  $\rightarrow$  Nat  $\rightarrow$  Nat
1108   Solves : Alg  $\rightarrow$  (Nat  $\rightarrow$  Nat  $\rightarrow$  Prop)  $\rightarrow$  Prop
1109
1110 /-- The factoring relation: p is a nontrivial divisor of n. -/
1111 def factorRel (n p : Nat) : Prop := 1 < p  $\wedge$  p < n  $\wedge \exists q, p * q = n$ 
1112
1113 def EasySearch (M : Machine) (R : Nat  $\rightarrow$  Nat  $\rightarrow$  Prop) : Prop :=
1114    $\exists A : M.\text{Alg}, M.\text{Solves } A \ R \ \text{Poly } (M.\text{time } A)$ 
1115
1116 /-- Prime freedom read as cost: no fast reader selects a point of the witness orbit.
1117 -/
1118 def PrimeHidden (M : Machine) : Prop :=  $\neg \text{EasySearch } M \ \text{factorRel}$ 
1119
1120 /-- The collapse field. 'PeqNP' is the class equality on this machine; 'search' is
1121 the

```

```

1110 classical search-to-decision theorem specialized to factoring, an NP relation. -/
1111 structure Collapse (M : Machine) where
1112   PegNP : Prop
1113   search : PegNP → EasySearch M factorRel
1114
1115 /-- THE PRIME BRIDGE. Hidden prime freedom yields the separation. -/
1116 theorem prime_bridge (M : Machine) (C : Collapse M) (hidden : PrimeHidden M) : ~
1117   C.PegNP :=
1118   fun h => hidden (C.search h)
1119
1120 /-- The Shor face: a coherent machine where the fibre is read fast. -/
1121 def revealing : Machine := (Unit, fun _ => 1, fun _ => True)
1122
1123 theorem one_is_poly : Poly (fun _ => 1) := (1, 0, fun n => Nat.le_add_left 1 (1 * n
1124   ^ 0))
1125
1126 /-- The posit is keyed: it fails on a coherent machine. -/
1127 theorem prime_hidden_is_keyed : ~ PrimeHidden revealing :=
1128   fun h => h ((), trivial, one_is_poly)
1129
1130 /-- Strictly stronger than the separation: on the revealing machine with the class
1131   equality false, the separation holds and the posit fails. -/
1132 def separatedButRevealing : Collapse revealing := (False, fun h => absurd h id)
1133
1134 theorem prime_bridge_is_one_way :
1135   ~ ∀ (M : Machine) (C : Collapse M), ~ C.PegNP → PrimeHidden M :=
1136   fun h => prime_hidden_is_keyed (h revealing separatedButRevealing id)
1137
1138 /-- ## V · the bridge, whole -/
1139 theorem the_prime_bridge :
1140   -- the fibre: four points, off the seat, swap-closed, one witness orbit
1141   (fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] ∧
1142     (fibre 15).all (fun p => swap p != p) = true ∧
1143     (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]) ∧
1144   -- F1 · the orientation bit is walled and gauge
1145   ((~ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5) ∧
1146     ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5), (3, 5)]) ∧
1147   -- F2 · the description fixes the witness: zero informational freedom
1148   (((fibre 15).filter (fun p => !trivialPt p)).map canon).eraseDups = [(3, 5)] ∧
1149   -- the prime bridge, its key, and its direction
1150   (∀ (M : Machine) (C : Collapse M), PrimeHidden M → ~ C.PegNP) ∧
1151   ~ PrimeHidden revealing ∧
1152   (∀ (M : Machine) (C : Collapse M), ~ C.PegNP → PrimeHidden M) :=
1153   ((fibre 15, off_seat 15, witness_orbit 15.1), (orientation_wall,
1154     orientation_is_gauge),
1155     witness_determined 15, prime_bridge, prime_hidden_is_keyed,
1156     prime_bridge_is_one_way)
1157
1158 end PNP.PRIMEBRIDGE
1159
1160 /-- ## Cones, pinned: every theorem axiom-free -/
1161 /-- info: 'PNP.PRIMEBRIDGE.fibre_15' does not depend on any axioms -/
1162 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.fibre_15
1163 /-- info: 'PNP.PRIMEBRIDGE.off_seat_15' does not depend on any axioms -/
1164 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.off_seat_15
1165 /-- info: 'PNP.PRIMEBRIDGE.swap_closed_15' does not depend on any axioms -/
1166 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.swap_closed_15
1167 /-- info: 'PNP.PRIMEBRIDGE.witness_orbit_15' does not depend on any axioms -/
1168 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_orbit_15
1169 /-- info: 'PNP.PRIMEBRIDGE.orientation_wall' does not depend on any axioms -/
1170 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_wall
1171 /-- info: 'PNP.PRIMEBRIDGE.canon_even' does not depend on any axioms -/
1172 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.canon_even
1173 /-- info: 'PNP.PRIMEBRIDGE.orientation_is_gauge' does not depend on any axioms -/
1174 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_is_gauge
1175 /-- info: 'PNP.PRIMEBRIDGE.description_determines_witnesses' does not depend on any
1176   axioms -/
1177 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.description_determines_witnesses
1178 /-- info: 'PNP.PRIMEBRIDGE.witness_determined_15' does not depend on any axioms -/
1179 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_determined_15
1180 /-- info: 'PNP.PRIMEBRIDGE.prime_bridge' does not depend on any axioms -/
1181 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge
1182 /-- info: 'PNP.PRIMEBRIDGE.one_is_poly' does not depend on any axioms -/
1183 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.one_is_poly
1184 /-- info: 'PNP.PRIMEBRIDGE.prime_hidden_is_keyed' does not depend on any axioms -/
1185 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_hidden_is_keyed
1186 /-- info: 'PNP.PRIMEBRIDGE.prime_bridge_is_one_way' does not depend on any axioms -/
1187 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge_is_one_way
1188 /-- info: 'PNP.PRIMEBRIDGE.the_prime_bridge' does not depend on any axioms -/
1189 #guard_msgs in #print axioms PNP.PRIMEBRIDGE.the_prime_bridge
1190
1191 /--
1192 # PNP.ROUND2 · the blocker of the prime bridge, located, and the second attack
1193
1194 ROUND 1 LEFT THE KEY ON 'PrimeHidden': no polynomial reader selects a point of the
1195   semiprime witness orbit. F2 proved the description carries the witness with zero
1196   informational freedom, so the key is a pure cost statement against readers that have
1197   everything. The blocker has three layers.
1198
1199   B1 · Relativization. A reader with a factoring oracle reads the orbit in one step.
1200     The posit fails relative to that oracle, so no argument valid relative to
1201     every
1202     oracle proves it (Baker–Gill–Solovay pattern).
1203   B2 · The self-gag. If the posit holds, factoring-based pseudorandom functions
1204     exist
1205     (Naor–Reingold), and then no natural property proves the circuit lower bound
1206     (Razborov–Rudich). The posit, if true, bars the natural route to itself.
1207   B3 · The floor points the wrong way. The black-box floor is proved unbreakable
1208     (no savings below 2^N for a test-only reader). Every known unconditional
1209     route
1210     to circuit lower bounds runs the other way: it needs savings below 2^N for
1211     white-box circuit SAT (Williams 2011, NEXP ≠ ACC^0).
1212
1213   ROUND 2 attacks through B3, the one exit the barriers leave open: algorithms to
1214     lower
1215     bounds. The kinetic floor's black-box face cannot be beaten, by theorem here; the
1216     route requires beating its white-box face, which is exactly the access F2 located.

```

```

1208 The route reaches NEXP ≠ C. Two gaps separate it from P ≠ NP, and both are named.
1209
1210 Cited theorems enter as structure fields, never as `axiom`. Core Lean 4, no import.
1211 Every theorem pinned at the foot: no axiom of any kind.
1212 -/
1213
1214 namespace PNP.ROUND2
1215
1216 /--! ## I · the black-box floor admits no savings (axiom-free pigeonhole) -/
1217 namespace FLOOR
1218
1219 def has : List Nat → Nat → Bool
1220 | [], _ => false
1221 | y :: ys, k => Nat.beq y k || has ys k
1222
1223 def drop : List Nat → Nat → List Nat
1224 | [], _ => []
1225 | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)
1226
1227 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1228 | 0 => rfl
1229 | n + 1 => beq_self n
1230
1231 theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
1232 | [], _ => Nat.le_refl 0
1233 | y :: ys, k => by
1234   unfold drop
1235   cases Nat.beq y k with
1236   | true => exact Nat.le_succ_of_le (drop_le ys k)
1237   | false => exact Nat.succ_le_succ (drop_le ys k)
1238
1239 theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length +
1240   1 ≤ l.length
1241 | [], _, h => Bool.noConfusion h
1242 | y :: ys, k, h => by
1243   unfold drop
1244   unfold has at h
1245   cases hy : Nat.beq y k with
1246   | true => exact Nat.succ_le_succ (drop_le ys k)
1247   | false =>
1248     rw [hy] at h
1249     exact Nat.succ_le_succ (drop_len ys k h)
1250
1251 theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
1252   has l x = true → has (drop l k) x = true
1253 | [], _, _, h => Bool.noConfusion h
1254 | y :: ys, k, x, hxx, h => by
1255   unfold drop
1256   unfold has at h
1257   cases hy : Nat.beq y k with
1258   | true =>
1259     cases hxx : Nat.beq y x with
1260     | true =>
1261       have x = k := (Nat.eq_of_beq_eq_true hxx).symm.trans
1262         (Nat.eq_of_beq_eq_true hy)
1263       rw [this, beq_self] at hxx; exact Bool.noConfusion hxx
1264     | false =>
1265       rw [hxx] at h
1266       exact has_drop ys k x hxx h
1267   | false =>
1268     show has (y :: drop ys k) x = true
1269     unfold has
1270     cases hxx : Nat.beq y x with
1271     | true => rfl
1272     | false =>
1273       rw [hxx] at h
1274       exact has_drop ys k x hxx h
1275
1276 theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
1277   cases hb : Nat.beq x K with
1278   | false => rfl
1279   | true =>
1280     have e : x = K := Nat.eq_of_beq_eq_true hb
1281     rw [e] at h; exact absurd h (Nat.lt_irrefl K)
1282
1283 theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤
1284   l.length
1285 | 0, _, _ => Nat.zero_le _
1286 | K + 1, l, h =>
1287   Nat.le_trans
1288     (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
1289       has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx)))))
1290     (drop_len l K (h K (Nat.lt_succ_self K)))
1291
1292 def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
1293
1294 theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has
1295   l x
1296 | [], _ => rfl
1297 | y :: ys, x => by
1298   show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
1299   rw [run_eq_has ys x]
1300
1301 /-- B3, the black-box face: no test-only reader saves a single query below 2^N. -/
1302 theorem black_box_no_savings (N : Nat) (path : List Nat)
1303   (correct : ∀ f : Nat → Bool, run path f = true → ∃ x, x < 2 ^ N ∧ f x = true) :
1304   ~ path.length < 2 ^ N :=
1305   Nat.not_lt_of_le (pigeon (2 ^ N) path (fun x hx => by
1306     rw [run_eq_has]
1307     exact (correct _).mpr (x, hx, beq_self x)))
1308
1309 end FLOOR
1310
1311 /--! ## II · B1 · relativization, typed -/
1312
1313 /-- An oracle world: the posit read relative to an oracle. `factoring` is the oracle
1314   that answers the witness orbit; relative to it the posit fails. -/

```

```

1311 structure Relativized where
1312   Oracle      : Type
1313   Hidden      : Oracle → Prop
1314   factoring    : Oracle
1315   easy        : ~ Hidden factoring
1316
1317 -- No argument valid relative to every oracle proves the posit. -/
1318 theorem b1_no_relativizing_proof (R : Relativized) : ~ ∀ 0, R.Hidden 0 :=
1319   fun h => R.easy (h R.factoring)
1320
1321 --! ## III · B2 · the self-gag, typed -/
1322
1323 -- The two cited links: hidden prime freedom gives pseudorandom functions
1324 (Naor-Reingold), and pseudorandom functions bar natural lower bounds
1325 (Razborov-Rudich). -/
1326 structure SelfGag where
1327   Hidden      : Prop
1328   PRF         : Prop
1329   NaturalLB   : Prop
1330   nr          : Hidden → PRF
1331   rr          : PRF → ~ NaturalLB
1332
1333 -- If the posit holds, no natural argument proves the lower bound it needs. -/
1334 theorem b2_self_gag (G : SelfGag) (h : G.Hidden) : ~ G.NaturalLB :=
1335   G.rr (G.nr h)
1336
1337 -- Equivalently: a natural proof of the lower bound would refute the posit. -/
1338 theorem b2_natural_refutes (G : SelfGag) (n : G.NaturalLB) : ~ G.Hidden :=
1339   fun h => b2_self_gag G h n
1340
1341 --! ## IV · round 2 · algorithms to lower bounds, the open exit -/
1342
1343 -- The Williams route, cited. 'Savings C' is a white-box SAT algorithm for circuit
1344 class C running below 2^n by a superpolynomial factor. 'williams' is the 2011
1345 theorem
1346 pattern: savings for C give NEXP ⊆ C. 'scaleDown' and 'generalize' are the two gaps
1347 to P ≠ NP: from NEXP down to NP, and from the restricted class C up to P/poly. -/
1348 structure Route where
1349   Class      : Type
1350   Savings    : Class → Prop
1351   NEXPnotIn  : Class → Prop
1352   williams   : ∀ C, Savings C → NEXPnotIn C
1353   Ppoly      : Class
1354   NPnotPpoly : Prop
1355   Sep        : Prop
1356   karp_sep   : NPnotPpoly → Sep
1357
1358 -- The route reaches NEXP ⊆ C on savings, and no further on its own fields. -/
1359 theorem round2_reaches (R : Route) (C : R.Class) (s : R.Savings C) : R.NEXPnotIn C
1360 :=
1361   R.williams C s
1362
1363 -- What closes the route: savings for general circuits, plus the scale-down from
1364 NEXP to NP. Supplied both, the separation follows. -/
1365 theorem round2_closes (R : Route) (s : R.Savings R.Ppoly)
1366 (scaleDown : R.NEXPnotIn R.Ppoly → R.NPnotPpoly) : R.Sep :=
1367   R.karp_sep (scaleDown (R.williams R.Ppoly s))
1368
1369 -- The inversion, whole: the black-box floor cannot be beaten; the lower-bound
1370 route
1371 needs the white-box floor beaten; the posit gags the natural route; the posit fails
1372 relative to the factoring oracle. -/
1373 theorem round2_terminal :
1374   (∀ (N : Nat) (path : List Nat),
1375     (∀ f : Nat → Bool, FLOOR.run path f = true → ∃ x, x < 2 ^ N ∧ f x = true) →
1376     ~ path.length < 2 ^ N) ∧
1377   (∀ R : Relativized, ~ ∀ 0, R.Hidden 0) ∧
1378   (∀ G : SelfGag, G.Hidden → ~ G.NaturalLB) ∧
1379   (∀ (R : Route) (C : R.Class), R.Savings C → R.NEXPnotIn C) ∧
1380   (∀ (R : Route), R.Savings R.Ppoly →
1381     (R.NEXPnotIn R.Ppoly → R.NPnotPpoly) → R.Sep) :=
1382   (FLOOR.black_box_no_savings, b1_no_relativizing_proof, b2_self_gag,
1383     round2_reaches, round2_closes)
1384
1385 end PNP.ROUND2
1386
1387 --! ## Cones, pinned: every theorem axiom-free -/
1388 -- info: 'PNP.ROUND2.FLOOR.beq_self' does not depend on any axioms -/
1389 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_self
1390 -- info: 'PNP.ROUND2.FLOOR.drop le' does not depend on any axioms -/
1391 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_le
1392 -- info: 'PNP.ROUND2.FLOOR.drop len' does not depend on any axioms -/
1393 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_len
1394 -- info: 'PNP.ROUND2.FLOOR.has_drop' does not depend on any axioms -/
1395 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.has_drop
1396 -- info: 'PNP.ROUND2.FLOOR.beq_false_of_lt' does not depend on any axioms -/
1397 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_false_of_lt
1398 -- info: 'PNP.ROUND2.FLOOR.pigeon' does not depend on any axioms -/
1399 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.pigeon
1400 -- info: 'PNP.ROUND2.FLOOR.run_eq_has' does not depend on any axioms -/
1401 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.run_eq_has
1402 -- info: 'PNP.ROUND2.FLOOR.black_box_no_savings' does not depend on any axioms -/
1403 #guard_msgs in #print axioms PNP.ROUND2.FLOOR.black_box_no_savings
1404 -- info: 'PNP.ROUND2.b1_no_relativizing_proof' does not depend on any axioms -/
1405 #guard_msgs in #print axioms PNP.ROUND2.b1_no_relativizing_proof
1406 -- info: 'PNP.ROUND2.b2_self_gag' does not depend on any axioms -/
1407 #guard_msgs in #print axioms PNP.ROUND2.b2_self_gag
1408 -- info: 'PNP.ROUND2.b2_natural_refutes' does not depend on any axioms -/
1409 #guard_msgs in #print axioms PNP.ROUND2.b2_natural_refutes
1410 -- info: 'PNP.ROUND2.round2_reaches' does not depend on any axioms -/
1411 #guard_msgs in #print axioms PNP.ROUND2.round2_reaches
1412 -- info: 'PNP.ROUND2.round2_closes' does not depend on any axioms -/
1413 #guard_msgs in #print axioms PNP.ROUND2.round2_closes
1414 -- info: 'PNP.ROUND2.round2_terminal' does not depend on any axioms -/
1415 #guard_msgs in #print axioms PNP.ROUND2.round2_terminal
1416
1417 --!

```

```

1414 # PNP.ROUTE · RA and Freedom as the only posits: where the route goes
1415
1416 Request: P ≠ NP with the Root Axiom and Freedom as the only posits, everything else
1417 axiom-free. This file decides whether that route exists.
1418
1419 I · RA is conservative. Its formal reading,  $\forall x : U, 0 < \Delta E x$ , is satisfied by
1420 interpreting U as an empty type. So any statement that does not mention U or
1421  $\Delta E$ 
1422 and is derived from RA is derived without it.
1423 II · Freedom is conservative. Its formal reading, a fibre of exactly two distinct
1424 orientations, is satisfied by Bool. The same conclusion follows.
1425 III · Hence a derivation of the separation from RA and Freedom, as written, is a
1426 derivation of the separation from nothing. The route through the posits as
1427 written is the route without them.
1428 IV · The one reading of RA that carries the load: read on computation, it says
1429 every decider of SAT pays superpolynomially many registrations. That reading
1430 yields the separation in one step and is equivalent to it. It is the bridge,
1431 in RA's vocabulary.
1432
1433 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1434 -/
1435
1436 namespace PNP.ROUTE
1437
1438 --! ## I · RA is conservative -/
1439
1440 -- The Root Axiom in the codex's own formal reading (ROOT.RA). -/
1441 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1442
1443 -- RA is satisfiable on any background: interpret U as the empty type. -/
1444 theorem ra_satisfiable : RA Empty (fun e => nomatch e) :=
1445   fun e => nomatch e
1446
1447 -- RA is conservative: whatever follows from RA under every interpretation of its
1448 symbols, and does not mention them, holds outright. -/
1449 theorem ra_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) :
1450   S :=
1451     h Empty (fun e => nomatch e) ra_satisfiable
1452
1453 --! ## II · Freedom is conservative -/
1454
1455 -- Freedom in the codex's formal reading: a fibre of exactly two distinct
1456 orientations. -/
1457 structure Freedom where
1458   Fibre : Type
1459   a : Fibre
1460   b : Fibre
1461   distinct : a ≠ b
1462   exhaust : ∀ x : Fibre, x = a ∨ x = b
1463
1464 -- Freedom is inhabited by the orientation bit itself. -/
1465 def boolFreedom : Freedom :=
1466   (Bool, true, false, Bool.noConfusion, fun x => match x with
1467     | true => 0r.inl rfl
1468     | false => 0r.inr rfl)
1469
1470 theorem freedom_conservative (S : Prop) (h : Freedom → S) : S := h boolFreedom
1471
1472 --! ## III · the two posits together add nothing -/
1473
1474 -- Any statement derived from RA and Freedom, uniformly in their interpretation,
1475 holds without them. -/
1476 theorem posits_add_nothing (S : Prop)
1477 (h : ∀ (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) : S :=
1478   h Empty (fun e => nomatch e) ra_satisfiable boolFreedom
1479
1480 --! ## IV · the one reading that carries the load -/
1481
1482 def Poly (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c
1483
1484 -- A machine model: algorithms, running time counted in registrations, and which
1485 algorithms decide SAT from its description. -/
1486 structure Machine where
1487   Alg : Type
1488   time : Alg → Nat → Nat
1489   decidesSAT : Alg → Prop
1490
1491 -- The separation on a machine model: no polynomial-time decider of SAT. -/
1492 def Sep (M : Machine) : Prop := ~ ∃ A : M.Alg, M.decidesSAT A ∧ Poly (M.time A)
1493
1494 -- RA read on computation: every decider of SAT actuates superpolynomially many
1495 registrations. -/
1496 def RAcomp (M : Machine) : Prop := ∀ A : M.Alg, M.decidesSAT A → ~ Poly (M.time A)
1497
1498 -- The separation, from RA read on computation. -/
1499 theorem sep_of_ra_comp (M : Machine) (h : RAcomp M) : Sep M :=
1500   fun (A, hA, hP) => h A hA hP
1501
1502 -- And conversely: the reading is the separation. -/
1503 theorem ra_comp_of_sep (M : Machine) (h : Sep M) : RAcomp M :=
1504   fun A hA hP => h (A, hA, hP)
1505
1506 theorem ra_comp_is_sep (M : Machine) : RAcomp M → Sep M :=
1507   (sep_of_ra_comp M, ra_comp_of_sep M)
1508
1509 -- RA read on computation is not conservative: it fails on a coherent machine. -/
1510 def fastMachine : Machine := (Unit, fun _ => 1, fun _ => True)
1511
1512 theorem ra_comp_is_keyed : ~ RAcomp fastMachine :=
1513   fun h => h () trivial (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))
1514
1515 --! ## V · the route, whole -/
1516
1517 theorem the_route :
1518   -- RA and Freedom as written add nothing to any statement outside their
1519   -- vocabulary
1520   (∀ S : Prop, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → Freedom → S) → S) ∧
1521   -- RA read on computation is exactly the separation

```

```

1517 (V M : Machine, RAcamp M ↔ Sep M) A
1518 -- and that reading is keyed: a coherent machine refutes it
1519 ~ RAcamp fastMachine :=
1520 (posits_add_nothing, ra_comp_is_sep, ra_comp_is_keyed)
1521
1522 end PNP.ROUTE
1523
1524 /-! ## Cones, pinned: every theorem axiom-free -/
1525 /-- info: 'PNP.ROUTE.ra_satisfiable' does not depend on any axioms -/
1526 #guard_msgs in #print axioms PNP.ROUTE.ra_satisfiable
1527 /-- info: 'PNP.ROUTE.ra_conservative' does not depend on any axioms -/
1528 #guard_msgs in #print axioms PNP.ROUTE.ra_conservative
1529 /-- info: 'PNP.ROUTE.freedom_conservative' does not depend on any axioms -/
1530 #guard_msgs in #print axioms PNP.ROUTE.freedom_conservative
1531 /-- info: 'PNP.ROUTE.posits_add_nothing' does not depend on any axioms -/
1532 #guard_msgs in #print axioms PNP.ROUTE.posits_add_nothing
1533 /-- info: 'PNP.ROUTE.sep_of_ra_comp' does not depend on any axioms -/
1534 #guard_msgs in #print axioms PNP.ROUTE.sep_of_ra_comp
1535 /-- info: 'PNP.ROUTE.ra_comp_of_sep' does not depend on any axioms -/
1536 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_of_sep
1537 /-- info: 'PNP.ROUTE.ra_comp_is_sep' does not depend on any axioms -/
1538 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_sep
1539 /-- info: 'PNP.ROUTE.ra_comp_is_keyed' does not depend on any axioms -/
1540 #guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_keyed
1541 /-- info: 'PNP.ROUTE.the_route' does not depend on any axioms -/
1542 #guard_msgs in #print axioms PNP.ROUTE.the_route
1543
1544 /-!
1545 # PNP.THREESEAL · the Tongue and the Form added for direction, with RA beneath
1546
1547 The Number is orientation-blind by theorem. The architecture's answer is to read
1548 direction from the other two seals: Seal L, the Tongue, which carries order, and
1549 Seal G, the Form, which carries the fold. This file runs all three on P versus NP
1550 and asks the one question that decides the matter: does any seal read the direction
1551 from the problem, or is the direction supplied?
1552
1553 I · Each seal is computed on the problem. The problem is the same sentence and
1554 the same fold in the equal world and in the separated world, so each seal's
1555 reading is world-constant: the Tongue seals both sentences alike, the Form
1556 finds no fixed locus for either, the Number reads one magnitude for both.
1557
1558 II · No function of the three readings returns the orientation. Adding seals adds
1559 readings, and a coalition of world-constant readings is world-constant.
1560
1561 III · RA is keyless and conservative, so it selects no orientation.
1562
1563 IV · Given one supplied odd bit at the seat, the lock forms and locks exactly one
1564 orientation: the calibration is unique. This is the lock locking only one.
1565
1566 V · The verdict: sealed at the act, premise grade on the supplied bit; without
1567 the act, the dot.
1568
1569 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1570 -/
1571
1572 namespace PNP.THREESEAL
1573
1574 inductive Token : Type
1575 | sealSep | sealEq | broken | opn | dot
1576 deriving DecidableEq, Repr
1577
1578 /-- The two worlds, indexed by the separation bit. -/
1579 abbrev World := Bool
1580
1581 /-! ## I · the three seals, computed on the problem -/
1582
1583 /-- Seal L. A sentence is three slots of literal ids; the Tongue seals it when the
1584 three slots are populated and pairwise disjoint. It checks form, not truth. -/
1585 def disjointL : List Nat → List Nat → Bool
1586 | [], _ => true
1587 | x :: xs, ys => !(ys.contains x) && disjointL xs ys
1588
1589 def seall (s : List (List Nat)) : Bool :=
1590 match s with
1591 | [a, b, c] => !a.isEmpty && !b.isEmpty && !c.isEmpty &&
1592 disjointL a b && disjointL a c && disjointL b c
1593 | _ => false
1594
1595 /-- "P is not NP" and "P is NP": same existence slot, same relation slot, the
1596 kinetic slot differing only in the negation literal. -/
1597 def sentSep : List (List Nat) := [[101], [201, 209], [301]]
1598 def sentEq : List (List Nat) := [[101], [201], [301]]
1599
1600 theorem tongue_seals_both : seall sentSep = true ∧ seall sentEq = true := by decide
1601
1602 /-- Seal G. The fold is complementation; its fixed set, counted. -/
1603 def foldFixed : Nat := ({false, true}.filter (fun b => (!b) == b)).length
1604
1605 theorem form_finds_no_locus : foldFixed = 0 := rfl
1606
1607 /-- Seal M. The lock scalar is the square of the signed scalar: reflection flips the
1608 sign and leaves the lock. -/
1609 def lockOf (signed : Int) : Int := signed * signed
1610
1611 theorem number_reads_one_magnitude : lockOf 1 = lockOf (-1) := by decide
1612
1613 /-- The three readings of the problem. The problem is fixed, so the readings do not
1614 depend on the world. -/
1615 def readings (w : World) : Bool × Nat × Int :=
1616 (seall sentSep && seall sentEq, foldFixed, lockOf 1)
1617
1618 theorem readings_world_constant : readings false = readings true := rfl
1619
1620 /-! ## II · no coalition of seals returns the orientation -/
1621 theorem seals_do_not_point :
1622 ~ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w :=
1623 fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (readings_world_constant →
1624 hg true))
1625
1626 /-! ## III · RA is keyless and selects nothing -/
1627 def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

```

```

1623 theorem ra_selects_nothing (S : Prop) (h : V (U : Type) (ΔE : U → Int), RA U ΔE → S)
1624 : S :=
1625 h Empty (fun e => nomatch e) (fun e => nomatch e)
1626
1627 /-! ## IV · given one supplied bit, the lock locks exactly one -/
1628
1629 /-- The orientation fibre: the two worlds, swapped by the denial. -/
1630 def deny (w : World) : World := !w
1631
1632 /-- The calibration: with a supplied odd witness `s` and the odd target `d`, exactly
1633 one calibration bit reconciles them on the pair. -/
1634 theorem locks_only_one (s d : World → Bool) (w : World)
1635 (hs : s (deny w) = !s w) (hd : d (deny w) = !d w) :
1636 ∃ c : Bool, (d w = xor (s w) c) ∧ d (deny w) = xor (s (deny w)) c) ∧
1637 ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') → c' = c := by
1638 refine (xor (d w) (s w), (?, ?), ?)
1639 · cases s w <|> cases d w <|> rfl
1640 · rw [hs, hd]; cases s w <|> cases d w <|> rfl
1641 · intro c' (h1, _)
1642 revert h1
1643 cases s w <|> cases d w <|> cases c' <|> intro h1 <|> first | rfl | exact
1644 Bool.noConfusion h1
1645
1646 /-! ## V · the verdict -/
1647
1648 /-- The act: the supplied bit. -/
1649 structure Act where
1650 bit : Bool
1651
1652 def verdict : Option Act → Token
1653 | none => .dot
1654 | some (true) => .sealSep
1655 | some (false) => .sealEq
1656
1657 theorem sealed_at_the_act :
1658 verdict (some (true)) = .sealSep ∧ verdict (some (false)) = .sealEq ∧ verdict
1659 none = .dot :=
1660 (rfl, rfl, rfl)
1661
1662 /-- Both acts are coherent: the lock forms under either supply, so the bit is spent,
1663 never derived. -/
1664 theorem both_acts_lock : verdict (some (true)) ≠ verdict (some (false)) := by decide
1665
1666 theorem three_seal_lock :
1667 (seall sentSep = true ∧ seall sentEq = true) ∧ foldFixed = 0 ∧ lockOf 1 = lockOf
1668 (-1) ∧
1669 (~ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w) ∧
1670 (V S : Prop, (V (U : Type) (ΔE : U → Int), RA U ΔE → S) → S) ∧
1671 (V (s d : World → Bool) (w : World), s (deny w) = !s w → d (deny w) = !d w →
1672 ∃ c : Bool, (d w = xor (s w) c) ∧ d (deny w) = xor (s (deny w)) c) ∧
1673 ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') → c' = c) ∧
1674 (verdict (some (true)) = .sealSep ∧ verdict none = .dot) :=
1675 (tongue_seals_both, form_finds_no_locus, number_reads_one_magnitude,
1676 seals_do_not_point,
1677 ra_selects_nothing, locks_only_one, (rfl, rfl))
1678
1679 end PNP.THREESEAL
1680
1681 /-! ## Cones, pinned: every theorem axiom-free -/
1682 /-- info: 'PNP.THREESEAL.tongue_seals_both' does not depend on any axioms -/
1683 #guard_msgs in #print axioms PNP.THREESEAL.tongue_seals_both
1684 /-- info: 'PNP.THREESEAL.form_finds_no_locus' does not depend on any axioms -/
1685 #guard_msgs in #print axioms PNP.THREESEAL.form_finds_no_locus
1686 /-- info: 'PNP.THREESEAL.number_reads_one_magnitude' does not depend on any axioms -/
1687 #guard_msgs in #print axioms PNP.THREESEAL.number_reads_one_magnitude
1688 /-- info: 'PNP.THREESEAL.readings_world_constant' does not depend on any axioms -/
1689 #guard_msgs in #print axioms PNP.THREESEAL.readings_world_constant
1690 /-- info: 'PNP.THREESEAL.seals_do_not_point' does not depend on any axioms -/
1691 #guard_msgs in #print axioms PNP.THREESEAL.seals_do_not_point
1692 /-- info: 'PNP.THREESEAL.ra_selects_nothing' does not depend on any axioms -/
1693 #guard_msgs in #print axioms PNP.THREESEAL.ra_selects_nothing
1694 /-- info: 'PNP.THREESEAL.locks_only_one' does not depend on any axioms -/
1695 #guard_msgs in #print axioms PNP.THREESEAL.locks_only_one
1696 /-- info: 'PNP.THREESEAL.sealed_at_the_act' does not depend on any axioms -/
1697 #guard_msgs in #print axioms PNP.THREESEAL.sealed_at_the_act
1698 /-- info: 'PNP.THREESEAL.both_acts_lock' does not depend on any axioms -/
1699 #guard_msgs in #print axioms PNP.THREESEAL.both_acts_lock
1700 /-- info: 'PNP.THREESEAL.three_seal_lock' does not depend on any axioms -/
1701 #guard_msgs in #print axioms PNP.THREESEAL.three_seal_lock
1702
1703 /-!
1704 # PNP.DEED · the kinetic pulse as the supplied bit
1705
1706 Proposal: supply is the kinetic pulse to examine and compute, and the compute itself
1707 is the supplied bit. This file tests it on the two locks that share the locus.
1708
1709 I · Every deed actuates: a registered compute has positive energy. The deed
1710 instantiates RA at itself. RA's lock is keyless and opens on the deed.
1711
1712 II · A deed's actuation does not certify its output. A compute that prints the
1713 separation and a compute that prints the equality carry the same energy, and
1714 both occur in either world. The pulse is world-constant.
1715
1716 III · The price is symmetric. Registering assent and registering denial each pay
1717 the
1718 floor once. For RA the denial instantiates the axiom it denies; for the
1719 separation the denial instantiates nothing about the separation.
1720
1721 IV · So the deed supplies RA's bit, never the separation's. The compute supplies
1722 its
1723 output, and whether that output is correct is the missing object again: a
1724 procedure whose correctness is proved.
1725
1726 Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.
1727 -/
1728
1729 namespace PNP.DEED
1730
1731 /-!

```

```

1722 abbrev World := Bool -- true: the separated world; false: the equal world
1723
1724 /-- A deed: a registered compute, with its energy and the bit it outputs. -/
1725 structure Deed where
1726   energy : Nat
1727   pos    : 0 < energy
1728   output : Bool
1729
1730 /-- The output is correct in a world when it matches that world's value. -/
1731 def correct (w : World) (d : Deed) : Prop := d.output = w
1732
1733 /--! ## I · every deed actuates: RA, keyless, opens on the deed -/
1734 theorem deed_instantiates_ra : ∀ d : Deed, 0 < d.energy := fun d => d.pos
1735
1736 /--! ## II · actuation does not certify the output -/
1737 def assent : Deed := (1, by decide, true)
1738 def denial : Deed := (1, by decide, false)
1739
1740 theorem same_pulse : assent.energy = denial.energy := rfl
1741
1742 /-- In every world both deeds occur, and exactly one of them is correct. -/
1743 theorem both_deeds_in_every_world (w : World) :
1744   0 < assent.energy ∧ 0 < denial.energy ∧ (correct w assent ∨ correct w denial) :=
1745   (assent.pos, denial.pos, match w with
1746   | true => 0r.inl rfl
1747   | false => 0r.inr rfl)
1748
1749 /-- No reading of the pulse decides correctness: energy does not separate a correct
1750 deed from an incorrect one. -/
1751 theorem pulse_does_not_certify :
1752   ¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output :=
1753   fun (_, hg) => Bool.noConfusion ((hg denial).symm.trans (hg assent))
1754
1755 /-- An incorrect deed actuates as fully as a correct one, in every world. -/
1756 theorem wrong_deeds_actuate (w : World) : ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy
1757 :=
1758   match w with
1759   | true => (denial, Bool.noConfusion, denial.pos)
1760   | false => (assent, Bool.noConfusion, assent.pos)
1761
1762 /--! ## III · the price is symmetric -/
1763 def price (floor : Nat) (d : Deed) : Nat := floor * (if d.output then 1 else 1)
1764
1765 theorem price_symmetric (floor : Nat) : price floor assent = price floor denial :=
1766   rfl
1767
1768 /-- RA's denial is itself a deed and instantiates RA; the separation's denial is a
1769 deed and instantiates RA too, which is no information about the separation. -/
1770 theorem denials_both_actuate : 0 < denial.energy ∧ 0 < assent.energy :=
1771   (denial.pos, assent.pos)
1772
1773 /--! ## IV · the deed supplies RA's bit, never the separation's -/
1774 theorem deed_supplies_ra_not_sep :
1775   (∀ d : Deed, 0 < d.energy) ∧
1776   (∀ w : World, ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy) ∧
1777   (¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output) ∧
1778   (∀ floor, price floor assent = price floor denial) :=
1779   (deed_instantiates_ra, wrong_deeds_actuate, pulse_does_not_certify,
1780    price_symmetric)
1781
1782 end PNP.DEED
1783
1784 /--! ## Cones, pinned: every theorem axiom-free -/
1785 /-- info: 'PNP.DEED.deed_instantiates_ra' does not depend on any axioms -/
1786 #guard_msgs in #print axioms PNP.DEED.deed_instantiates_ra
1787 /-- info: 'PNP.DEED.same_pulse' does not depend on any axioms -/
1788 #guard_msgs in #print axioms PNP.DEED.same_pulse
1789 /-- info: 'PNP.DEED.both_deeds_in_every_world' does not depend on any axioms -/
1790 #guard_msgs in #print axioms PNP.DEED.both_deeds_in_every_world
1791 /-- info: 'PNP.DEED.pulse_does_not_certify' does not depend on any axioms -/
1792 #guard_msgs in #print axioms PNP.DEED.pulse_does_not_certify
1793 /-- info: 'PNP.DEED.wrong_deeds_actuate' does not depend on any axioms -/
1794 #guard_msgs in #print axioms PNP.DEED.wrong_deeds_actuate
1795 /-- info: 'PNP.DEED.price_symmetric' does not depend on any axioms -/
1796 #guard_msgs in #print axioms PNP.DEED.price_symmetric
1797 /-- info: 'PNP.DEED.denials_both_actuate' does not depend on any axioms -/
1798 #guard_msgs in #print axioms PNP.DEED.denials_both_actuate
1799 /-- info: 'PNP.DEED.deed_supplies_ra_not_sep' does not depend on any axioms -/
1800 #guard_msgs in #print axioms PNP.DEED.deed_supplies_ra_not_sep
1801
1802 /--!
1803 # PNP.IDCHECK · is least escape the freedom of prime, or the arrow of RA?
1804
1805 The test is the architecture's own discriminator (Bridge_Final.lean): a keyless
1806 property holds on every frame; a keyed one has a frame inhabiting its denial; no
1807 property is both. Least escape is keyed. Prime freedom, the arrow and RA are
1808 keyless.
1809 So none of them is least escape, on any reading that keeps their content.
1810 Core Lean 4, no import, no axiom of any kind.
1811 -/
1812 namespace PNP.IDCHECK
1813
1814 structure Machine where
1815   Alg : Type
1816   Inst : Type
1817   size : Inst → Nat
1818   out : Alg → Inst → Bool
1819   time : Alg → Inst → Nat
1820   sat : Inst → Bool
1821
1822 def bound (c k n : Nat) : Nat := c * n ^ k + c
1823 def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=

```

```

1820   M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
1821 def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
1822
1823 theorem beq_self : ∀ n : Nat, Nat.beq n n = true
1824 | 0 => rfl
1825 | n + 1 => beq_self n
1826
1827 def closedWorld : Machine := (Nat, Unit, fun _ => 0, fun _ => false, fun _ => 0,
1828   fun _ => true)
1829 def escapeWorld : Machine := (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A 0, fun _
1830   => 0, fun _ => true)
1831
1832 theorem closed_closes : NothingEscapes closedWorld :=
1833   fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
1834 theorem escape_fails : ¬ NothingEscapes escapeWorld :=
1835   fun h => match h (0 : Nat) 0 0 with
1836   | ((), hx) => hx (beq_self 0, Nat.zero_le _)
1837
1838 /-- Least escape is keyed: true on one coherent machine, false on another. -/
1839 theorem least_escape_is_keyed : NothingEscapes closedWorld ∧ ¬ NothingEscapes
1840   escapeWorld :=
1841   (closed_closes, escape_fails)
1842
1843 /-- THE CHECK. No statement that reads the same on every machine is least escape. -/
1844 theorem no_keyless_statement_is_least_escape (P : Prop) :
1845   ¬ ∀ M : Machine, (P ↔ NothingEscapes M) :=
1846   fun h => escape_fails ((h escapeWorld).mp ((h closedWorld).mpr closed_closes))
1847
1848 /--! ## the three candidates, each keyless -/
1849
1850 /-- Prime freedom: the fibre of multiplication over 2 is exactly the pair and its
1851 reverse. -/
1852 def primeFreedom2 : Prop := ∀ a b : Nat, a < 3 → b < 3 → (a * b = 2 ↔ (a = 1 ∧ b =
1853   2) ∨ (a = 2 ∧ b = 1))
1854
1855 /-- The arrow: an identity exists on every type. -/
1856 def arrowOfRA : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a
1857 /-- RA in its formal reading, at any interpretation of its symbols. -/
1858 def raAt (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x
1859
1860 theorem prime_freedom_is_not_least_escape : ¬ ∀ M : Machine, (primeFreedom2 ↔
1861   NothingEscapes M) :=
1862   no_keyless_statement_is_least_escape _
1863 theorem arrow_is_not_least_escape : ¬ ∀ M : Machine, (arrowOfRA ↔ NothingEscapes M)
1864 :=
1865   no_keyless_statement_is_least_escape _
1866 theorem ra_is_not_least_escape (U : Type) (ΔE : U → Int) :
1867   ¬ ∀ M : Machine, (raAt U ΔE ↔ NothingEscapes M) :=
1868   no_keyless_statement_is_least_escape _
1869
1870 /-- What does carry least escape: a statement that varies with the machine, as least
1871 escape does. Any such statement that implies it on every machine is keyed in turn.
1872 -/
1873 theorem carrier_must_be_keyed (Q : Machine → Prop) (h : ∀ M, Q M → NothingEscapes M)
1874 :
1875   ¬ Q escapeWorld :=
1876   fun q => escape_fails (h escapeWorld q)
1877
1878 end PNP.IDCHECK
1879
1880 /-- info: 'PNP.IDCHECK.least_escape_is_keyed' does not depend on any axioms -/
1881 #guard_msgs in #print axioms PNP.IDCHECK.least_escape_is_keyed
1882 /-- info: 'PNP.IDCHECK.no_keyless_statement_is_least_escape' does not depend on any
1883   axioms -/
1884 #guard_msgs in #print axioms PNP.IDCHECK.no_keyless_statement_is_least_escape
1885 /-- info: 'PNP.IDCHECK.prime_freedom_is_not_least_escape' does not depend on any
1886   axioms -/
1887 #guard_msgs in #print axioms PNP.IDCHECK.prime_freedom_is_not_least_escape
1888 /-- info: 'PNP.IDCHECK.arrow_is_not_least_escape' does not depend on any axioms -/
1889 #guard_msgs in #print axioms PNP.IDCHECK.arrow_is_not_least_escape
1890 /-- info: 'PNP.IDCHECK.ra_is_not_least_escape' does not depend on any axioms -/
1891 #guard_msgs in #print axioms PNP.IDCHECK.ra_is_not_least_escape
1892 /-- info: 'PNP.IDCHECK.carrier_must_be_keyed' does not depend on any axioms -/
1893 #guard_msgs in #print axioms PNP.IDCHECK.carrier_must_be_keyed
1894
1895 /--! ## helper cones, pinned -/
1896 /-- info: 'PNP.MISSING.beq_self' does not depend on any axioms -/
1897 #guard_msgs in #print axioms PNP.MISSING.beq_self
1898 /-- info: 'PNP.IDCHECK.beq_self' does not depend on any axioms -/
1899 #guard_msgs in #print axioms PNP.IDCHECK.beq_self
1900 /-- info: 'PNP.IDCHECK.closed_closes' does not depend on any axioms -/
1901 #guard_msgs in #print axioms PNP.IDCHECK.closed_closes
1902 /-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms -/
1903 #guard_msgs in #print axioms PNP.IDCHECK.escape_fails

```

Author’s Provenance and Method Disclosure

The arc was found within Trisduction, the author’s verification architecture, through its operating system PhysOST[†] (Islam 2026c), and compiled with the assistance of an AI scribe; every theorem stands on its compiled proof, reproducible by any reader from the appendices. The author declares no competing interests. Correspondence: islammm@alumni.iu.edu.